

KAYAK TRAVEL SIMULATION SYSTEM

DATA 236 - Distributed Systems | Group 3

Savitha Vijayarangana (018315986), Sujith Dugyala (018322694), Lakshmi Bharathy Kumar (017613414), Liza Bharatkumar Lad (018292703), Daniel Kim (014641497), Anshika Goel (018317819)

Overview

This project simulates a miniature version of the Kayak travel platform by designing and implementing a distributed, three-tier application capable of handling real-world travel search and booking workflows. The system models core entities such as users, flights, hotels, cars, billing records, and administrators, each backed by appropriate relational or document-based database schemas. Using REST services and Kafka-based messaging as the backbone of the middleware layer, the application supports end-to-end operations including searching and filtering listings, making bookings, processing payments, and generating administrative analytics dashboards. On the client side, users interact with a functional interface that mirrors Kayak-style interactions, while the backend coordinates request routing, data processing, and inter-service communication within a scalable distributed architecture. Overall, the project emphasizes practical system design—tying together service-oriented principles, reliable data management, and responsive client-side workflows to emulate a real travel metasearch experience.

GitHub: <https://github.com/IamSavitha/kayak-simulation>

Objectives of the System

- Provide a unified travel booking experience for users
- Offer separate privilege-controlled portals for users and admins
- Deliver modular and scalable backend services
- Ensure high performance using caching and event streaming
- Utilize AI-based itinerary generation
- Offer real-time analytics for administrators
- Achieve cloud-native deployment using Docker & Kubernetes

Features Summary

User Features

- Login, signup, logout
- Search and book cars, hotels, and flights
- View booking history
- Receive AI-generated travel itineraries
- Manage user profiles

Admin Features

- Login, signup, logout
- Add/Edit/Delete cars, hotels, flights, and users
- View system-wide analytics dashboard
- Monitor events and booking trends

System Features

- Microservices communication via Kafka
- Redis caching for performance optimization
- Scalable Kubernetes deployment
- Database separation for structured (MySQL) and unstructured (MongoDB) data
- Automated container orchestration
- CI/CD deployment readiness

Technology Stack

Frontend Technologies

React

React is used to build both the User Portal and the Admin Portal. Benefits:

- Component-based development
- Efficient virtualization using Virtual DOM
- Fast UI rendering for dynamic travel search pages
- Integration-friendly with REST APIs
- Supports SPA (Single Page Application) routing

Backend Technologies

Node.js

A high-performance JavaScript runtime used to execute backend business logic.

Express.js

The main web framework used to create RESTful API endpoints. Each microservice runs as its own Express server:

- User Service
- Admin Service
- Car Service
- Hotel Service

- Flights Service

Benefits:

- Lightweight and fast
- Perfect for microservices
- Large ecosystem of middleware
- Easy integration with databases

AI Layer

FastAPI

FastAPI is used to build the AI Service, which:

- Reads user history from MongoDB
- Processes bookings
- Generates custom itineraries
- Returns results via REST responses

Benefits:

- Extremely fast (Starlette + Pydantic)
- Ideal for ML/AI inference
- Automatic OpenAPI schema generation

Databases

MySQL

Used for structured, transactional data:

- Bookings
- Users
- Inventory (cars, hotels, flights)
- Payment metadata (if added later)

Benefits:

- Strong ACID compliance
- Excellent for transactional integrity

MongoDB

Used for:

- User reviews
- Search history
- AI context data

Benefits:

- Schema-less data
- Great for dynamic, nested user metadata
- Works naturally with the AI service

Infrastructure Tools

Docker

All components—including frontend, backend services, and the AI service—are packaged as Docker containers. Benefits:

- Portable deployments
- Consistent environment across development, testing, and production
- Lightweight and fast

Kubernetes (K8s)

Used for orchestration of all microservices. Responsibilities:

- Auto scaling
- Load distribution
- Pod management
- Rolling updates
- Fault tolerance

Cloud Platform

Amazon Web Services (AWS)

The system is deployed using:

- AWS ALB: Traffic routing to the Kubernetes cluster
- EC2 / EKS: Running Kubernetes nodes
- S3 or CloudFront: Optional hosting for static assets
- IAM: Configurable security and role-based access

Communication Layer

Kafka Message Broker

Kafka is used for event-driven communication between microservices. Events include:

- BookingCreated

- BookingUpdated
- InventoryChanged
- UserSignedUp
- AdminOperations

Benefits:

- Loose coupling between services
- High-throughput communication
- Great for analytics and dashboards

Performance Optimization

Redis

Redis acts as an in-memory cache for:

- Frequently accessed listings
- User session tokens
- Search results
- Inventory lookups

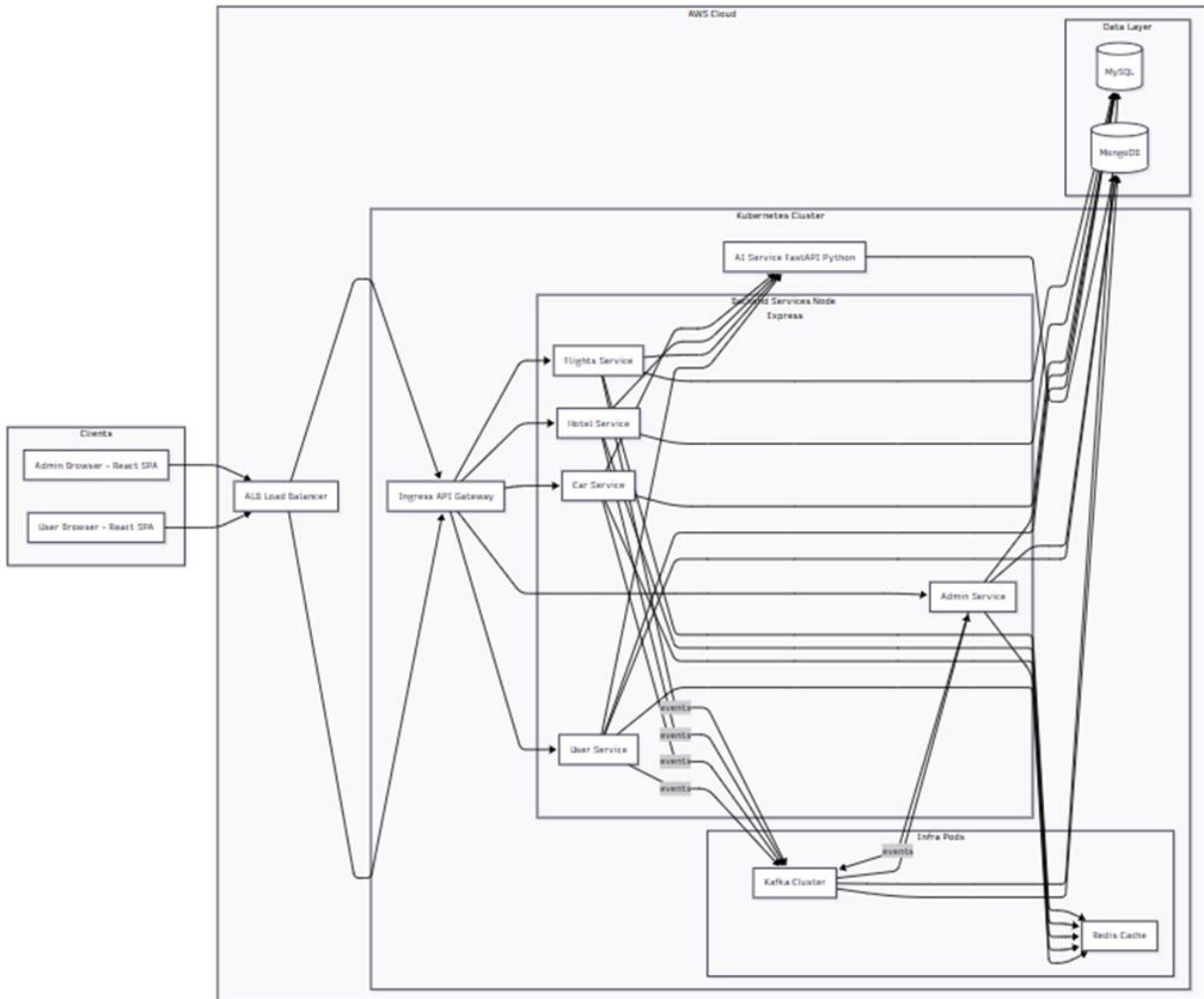
Benefits:

- Millisecond read/write
- Reduces database load
- Increases platform responsiveness

System Architecture Overview

A high-level architecture includes:

- React Frontend for users and admins
- API Gateway → Kubernetes Ingress
- Multiple Node.js microservices
- AI FastAPI service
- Kafka for inter-service event streaming
- Redis for caching
- MySQL & MongoDB for data storage
- AWS Load Balancer + EKS Cluster for cloud orchestration



Workflow Summary

User flow:

1. User logs in via React frontend
2. API Gateway routes to User Service
3. User searches for hotels/cars/flights
4. Results processed via services + cache
5. Booking stored in MySQL
6. Booking event emitted to Kafka
7. AI Service optionally generates an itinerary
8. User views dashboard and booking history

Admin flow:

1. Admin logs in through admin portal

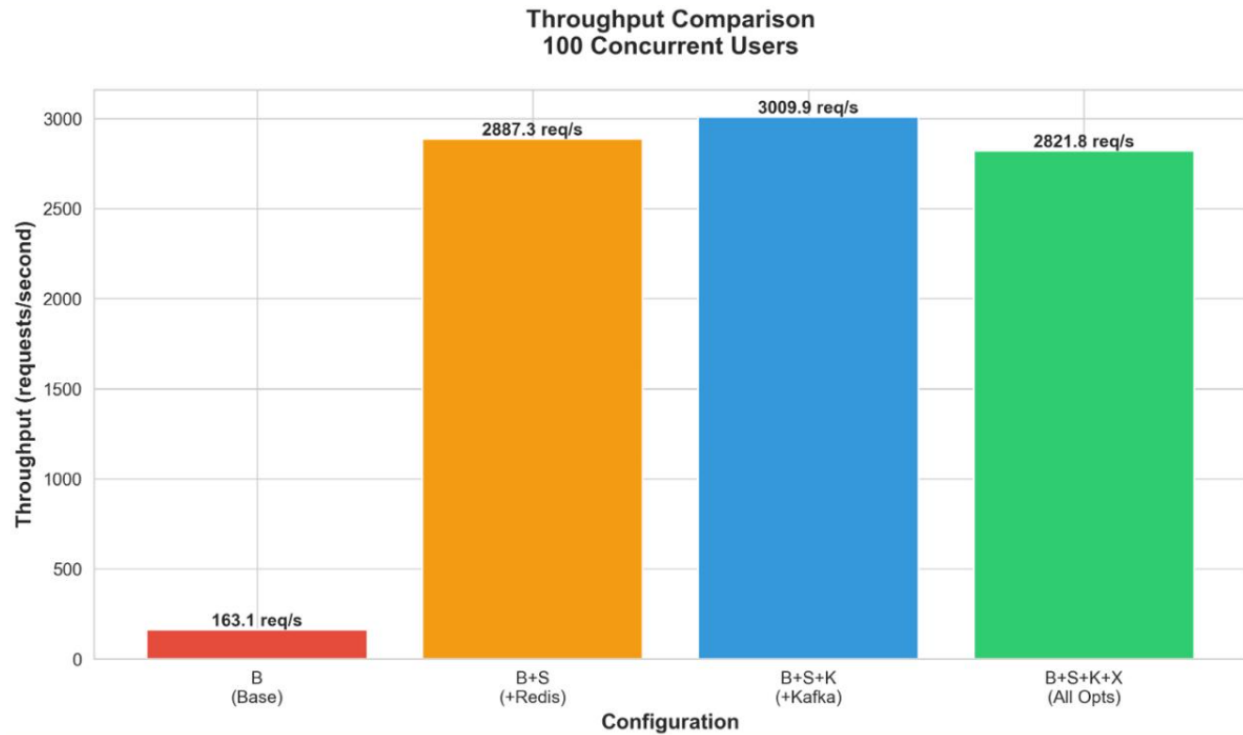
- 2. Admin Service validates privileges
- 3. Admin modifies cars/hotels/flights/users
- 4. CRUD operations sync with MySQL/MongoDB
- 5. Kafka events update the analytics engine
- 6. Admin views analytics dashboard

Performance & Load Testing Using Apache JMeter

Apache JMeter is used in this project to evaluate the performance, scalability, and stability of the microservices deployed within the Kubernetes cluster. Since the platform handles multiple user operations—such as searching hotels, booking flights, invoking AI itinerary services, and performing admin CRUD operations—load testing helps ensure that all services respond efficiently under high traffic. The objective of JMeter testing is to simulate real-world concurrent user activity and measure how the system behaves under various load conditions.

Performance Test Summary - 100 Concurrent Users

Configuration	Total Requests	Error Rate (%)	Avg Response Time (ms)	95th %ile (ms)	Throughput (req/s)	Status
B (Base)	49,014	0.00	582	1765	163.1	✅ PASS
B+S (+Redis)	866,192	0.00	32	93	2887.3	✅ PASS
B+S+K (+Kafka)	903,019	0.00	31	122	3009.9	✅ PASS
B+S+K+X (All Opts)	846,580	0.00	33	94	2821.8	✅ PASS



Amazon Web Services (AWS)

The screenshot displays the AWS Management Console interface for a target group named 'tg-booking-8000'. The left sidebar shows the navigation menu with categories like Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area shows the details of the target group, including its ARN, target type (IP), IP address type (IPv4), protocol (HTTP), port (8000), protocol version (HTTP1), and VPC (vpc-070fdbf8ed05aa1f6). Below the details, there is a table showing the distribution of targets by Availability Zone (AZ), with 1 total target, 1 healthy, 0 unhealthy, 0 unused, 0 initial, and 0 draining. The 'Targets' tab is selected, showing a list of registered targets (1) and options for anomaly mitigation, deregistering, and registering targets.

aws

Search

[Alt+S]

United States (Ohio)

Account ID: 4934-0481-9416

Bharathy_c

EC2 > Target groups > tg-booking-8000

tg-booking-8000

Actions

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

Load Balancing

Load Balancers

Target Groups

Trust Stores

Auto Scaling

Auto Scaling Groups

Settings

Details

arn:aws:elasticloadbalancing:us-east-2:493404819416:targetgroup/tg-booking-8000/c1b25b110f04f1b8

Target type IP	Protocol : Port HTTP: 8000	Protocol version HTTP1	VPC vpc-070fdbf8ed05aa1f6
IP address type IPv4	Load balancer kayak-balance-8000		

1	1 Healthy	0 Unhealthy	0 Unused	0 Initial	0 Draining
Total targets					
	0 Anomalous				

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

TargetsMonitoringHealth checksAttributesTags

Registered targets (1) Info

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target

aws

Search

[Alt+S]

United States (Ohio)

Account ID: 4934-0481-9416

Bharathy_c

EC2 > Target groups > tg-booking-8000

tg-booking-8000

Actions

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

Load Balancing

Load Balancers

Target Groups

Trust Stores

Auto Scaling

Auto Scaling Groups

Settings

Details

arn:aws:elasticloadbalancing:us-east-2:493404819416:targetgroup/tg-booking-8000/c1b25b110f04f1b8

Target type IP	Protocol : Port HTTP: 8000	Protocol version HTTP1	VPC vpc-070fdbf8ed05aa1f6
IP address type IPv4	Load balancer kayak-balance-8000		

1	1 Healthy	0 Unhealthy	0 Unused	0 Initial	0 Draining
Total targets					
	0 Anomalous				

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

TargetsMonitoringHealth checksAttributesTags

Registered targets (1) Info

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target

aws

Search

[Alt+S]

United States (Ohio)

Account ID: 4934-0481-9416

Bharathy_c

EC2 > Target groups > tg-booking-8000

tg-booking-8000

Actions

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

Load Balancing

Load Balancers

Target Groups

Trust Stores

Auto Scaling

Auto Scaling Groups

Settings

Details

arn:aws:elasticloadbalancing:us-east-2:493404819416:targetgroup/tg-booking-8000/c1b25b110f04f1b8

Target type IP	Protocol : Port HTTP: 8000	Protocol version HTTP1	VPC vpc-070fdbf8ed05aa1f6
IP address type IPv4	Load balancer kayak-balance-8000		

1	1 Healthy	0 Unhealthy	0 Unused	0 Initial	0 Draining
Total targets	0 Anomalous				

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

TargetsMonitoringHealth checksAttributesTags

Registered targets (1)

Info

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target

aws

Search

[Alt+S]

United States (Ohio)

Account ID: 4934-0481-9416

Bharathy_c

EC2 > Target groups > tg-booking-8000

tg-booking-8000

Actions

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

Load Balancing

Load Balancers

Target Groups

Trust Stores

Auto Scaling

Auto Scaling Groups

Settings

Details

arn:aws:elasticloadbalancing:us-east-2:493404819416:targetgroup/tg-booking-8000/c1b25b110f04f1b8

Target type IP	Protocol : Port HTTP: 8000	Protocol version HTTP1	VPC vpc-070fdbf8ed05aa1f6
IP address type IPv4	Load balancer kayak-balance-8000		

1	1 Healthy	0 Unhealthy	0 Unused	0 Initial	0 Draining
Total targets	0 Anomalous				

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

TargetsMonitoringHealth checksAttributesTags

Registered targets (1)

Info

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target

Functional Requirement

1. User Module (Customer-Facing Functionality)

Account & Profile

- User registration
- User login (JWT-based)
- Profile management
- Session handling via Redis
- View past bookings
- View upcoming trips

Search & Browsing

- Search flights (origin, destination, date, passengers)
- Search hotels (city, dates, guests, amenities)
- Search rental cars (location, dates)
- Sort & filter (price, rating, stops, amenities, distance)

Booking

- Select flight/hotel room/car
- Price calculation
- Check availability (via MySQL)
- Prevent double booking (date overlap logic)
- Booking confirmation

Payment

- Initiate payment
- Payment status updates
- Invoice generated and stored (Billing table)

- Email confirmation via Kafka Event

2. Admin Module (Management Functionality)

Admin Authentication

- Admin login
- Role-based access control

Inventory Management

- Create/update/delete flights
- Create/update/delete hotels
- Manage hotel rooms
- Manage car listings
- Update prices, availability, amenities

User & Booking Management

- View user list
- View bookings
- Manually update booking status
- Cancel/refund bookings

Audit Logging (MongoDB)

- Track every admin action
- Log old vs new values
- Store IP, endpoint, timestamp

3. Flight Module Functionality

- Flight search (date, airports, passengers)
- Display flight details (price, availability, ratings)

- Flight booking
- Decrease available seats after booking
- Store/track flight ratings
- Real-time price updates

4. Hotel & Room Module Functionality

Hotels

- Search hotel by location
- Filter by amenities, price, rating
- View hotel details

Rooms

- Fetch room availability
- Fetch room price
- Prevent overlapping bookings
- Store amenities per room

Booking Workflow

- Date selection
- Pricing per night × nights
- Confirmation

5. Car Rental Module Functionality

- Search cars by location + pickup/drop dates
- View car providers, vehicle details
- Price per day calculation
- Availability check
- Car booking confirmation

6. AI Concierge Module (Personalization)

Personalized Travel Assistant

- Suggest flights/hotels/cars based on preferences
- Recommend trip bundles (flight + hotel + car)
- Use user preferences, clickstream, past bookings

Chatbot

- AI-powered travel chat
- Stores conversation history in MongoDB
- Answers travel questions
- Suggests deals
- Provides trip plans

7. Deals & Price Watch Module

Deals Engine

- Detect price drops from price_history
- Identify high-discount listings
- AI calculates deal_score + urgency_score
- Store deals in MongoDB
- Show deals on homepage

Price Watch

- User sets alert criteria
- System triggers notifications when threshold met
- Uses Kafka to send event to Notification Service

8. Search Optimization via Redis

- Cache flight/hotel/car search results

- Serve results instantly on repeated queries
- Cache TTL logic
- Reduce database load

9. Event-Driven Kafka Functionality

Event Publishing

- user.created
- booking.created
- booking.updated
- payment.completed
- search events
- analytics.pageview
- deals.detected

Consumers

- AI Service → real-time personalization
- Analytics Service → dashboard
- Email Service → send booking/payment confirmations

This enables **decoupled, scalable microservices**.

10. MongoDB Analytics Functionality

Clickstream Logging

- Store every click/search/filter action

User Preferences

- Build a preference profile automatically

Price History

- Track price fluctuations over time

Reviews

- Store reviews & images

Admin Audit Logs

- Trace all admin updates

11. Payment & Billing Functionality

- Payment initiation
- Payment success/failure events
- Invoice generation
- Tax calculation
- Payment method storage (last 4 digits)

All stored in **MySQL Billing table**, ensuring transactional safety.

12. Notification Functionality

- Booking confirmation email
- Payment confirmation email
- Price drop alerts
- Deal alerts
- Admin notifications

Triggered via Kafka.

13. System Infrastructure Functionality

Docker

- Containerized microservices

Kubernetes

- Deployment
- Service scaling
- Load balancing
- Rolling updates

AWS

Hosting

Storage

Compute

TESTING PLAN

1. Start All Services

`docker-compose up -d`

```
kayak:user:updated
(base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % docker-compose ps
```

NAME	PORTS	IMAGE	COMMAND	SERVICE	CREATED	STATUS
kayak_admin_service		kayak-simulation-admin-service	"/app/start_service..."	admin-service	About an hour ago	Up About an h
our (healthy)	0.0.0.0:8006->8000/tcp, [::]:8006->8000/tcp					
kayak_ai_service		kayak-simulation-ai-service	"uvicorn ai_service..."	ai-service	About an hour ago	Up About an h
our (healthy)	0.0.0.0:8008->8000/tcp, [::]:8008->8000/tcp					
kayak_billing_service		kayak-simulation-billing-service	"/app/start_service..."	billing-service	About an hour ago	Up About an h
our (healthy)	0.0.0.0:8005->8000/tcp, [::]:8005->8000/tcp					
kayak_booking_service		kayak-simulation-booking-service	"/app/start_service..."	booking-service	16 minutes ago	Up 16 minutes
(healthy)	0.0.0.0:8009->8000/tcp, [::]:8009->8000/tcp					
kayak_car_service		kayak-simulation-car-service	"/app/start_service..."	car-service	About an hour ago	Up About an h
our (healthy)	0.0.0.0:8004->8000/tcp, [::]:8004->8000/tcp					
kayak_email_service		kayak-simulation-email-service	"/app/start_service..."	email-service	16 minutes ago	Up 16 minutes
(healthy)	0.0.0.0:8010->8000/tcp, [::]:8010->8000/tcp					
kayak_flight_service		kayak-simulation-flight-service	"/app/start_service..."	flight-service	About an hour ago	Up About an h
our (healthy)	0.0.0.0:8002->8000/tcp, [::]:8002->8000/tcp					
kayak_frontend		kayak-simulation-frontend	"docker-entrypoint.s..."	frontend	About an hour ago	Up 48 minutes
	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp					
kayak_hotel_service		kayak-simulation-hotel-service	"/app/start_service..."	hotel-service	About an hour ago	Up 7 minutes
(healthy)	0.0.0.0:8003->8000/tcp, [::]:8003->8000/tcp					
kayak_kafka		confluentinc/cp-kafka:7.5.0	"/etc/confluent/dock..."	kafka	About an hour ago	Up About an h
our (healthy)	0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp, 0.0.0.0:29092->29092/tcp, [::]:29092->29092/tcp					
kayak_kafka_ui		provectuslabs/kafka-ui:latest	"/bin/sh -c 'java --..."	kafka-ui	About an hour ago	Up 16 minutes
	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp					
kayak_mongodb		mongo:6.0	"docker-entrypoint.s..."	mongodb	About an hour ago	Up About an h
our (healthy)	0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp					
kayak_mysql		mysql:8.0	"docker-entrypoint.s..."	mysql	About an hour ago	Up About an h
our (healthy)	0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp					
kayak_redis		redis:7-alpine	"docker-entrypoint.s..."	redis	About an hour ago	Up About an h
our (healthy)	0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp					
kayak_search_service		kayak-simulation-search-service	"/app/start_service..."	search-service	About an hour ago	Up About an h
our (healthy)	0.0.0.0:8007->8000/tcp, [::]:8007->8000/tcp					
kayak_user_service		kayak-simulation-user-service	"/app/start_service..."	user-service	About an hour ago	Up About an h
our (healthy)	0.0.0.0:8001->8000/tcp, [::]:8001->8000/tcp					
kayak_zookeeper		confluentinc/cp-zookeeper:7.5.0	"/etc/confluent/dock..."	zookeeper	About an hour ago	Up About an h
our (healthy)	0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp					

```
(base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation %
```

2. Verify Services Health

Check service health

```
curl http://localhost:8001/health # User Service
curl http://localhost:8002/health # Flight Service
curl http://localhost:8003/health # Hotel Service
curl http://localhost:8004/health # Car Service
curl http://localhost:8005/health # Billing Service
curl http://localhost:8006/health # Admin Service
curl http://localhost:8007/health # Search Service
curl http://localhost:8008/health # AI Service
curl http://localhost:8009/health # Booking Service
curl http://localhost:8010/health # Email Service
```

```

● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl http://localhost:8001/health
{"status":"healthy","service":"user-service"}%
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl http://localhost:8002/health
{"status":"healthy","service":"flight-service"}%
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl http://localhost:8002/health
{"status":"healthy","service":"flight-service"}%
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl http://localhost:8004/health
{"status":"healthy","service":"car-service"}%
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl http://localhost:8005/health
{"status":"healthy","service":"billing-service"}%
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl http://localhost:8006/health
{"status":"healthy","service":"admin-service"}%
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl http://localhost:8007/health
{"status":"healthy","service":"search-service"}%
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl http://localhost:8008/health
{"status":"healthy","service":"ai-recommendation-service"}%
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl http://localhost:8009/health
{"status":"healthy","service":"booking-service"}%
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl http://localhost:8010/health
{"status":"healthy","service":"email-service"}%

```

3. Check Database Connections

MySQL

```

● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % docker-compose exec mysql mysql -u kayak_user -pkayak_pass kayak_db
-e "SHOW TABLES;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| Tables_in_kayak_db |
+-----+
| admins              |
| billings             |
| bookings            |
| cars                |
| flights             |
| hotel_rooms         |
| hotels              |
| users               |
+-----+

```

MongoDB

```

● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % docker-compose exec mongodb mongosh -u admin -p kayak_mongo_pass --authenticationDatabase admin --eval "db.adminCommand('listDatabases')"
{
  databases: [
    { name: 'admin', sizeOnDisk: Long('102400'), empty: false },
    { name: 'config', sizeOnDisk: Long('12288'), empty: false },
    { name: 'kayak_db', sizeOnDisk: Long('184320'), empty: false },
    { name: 'local', sizeOnDisk: Long('73728'), empty: false }
  ],
  totalSize: Long('372736'),
  totalSizeMb: Long('0'),
  ok: 1
}
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation %

```

Redis

```

● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % docker-compose exec redis redis-cli ping
PONG
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation %

```

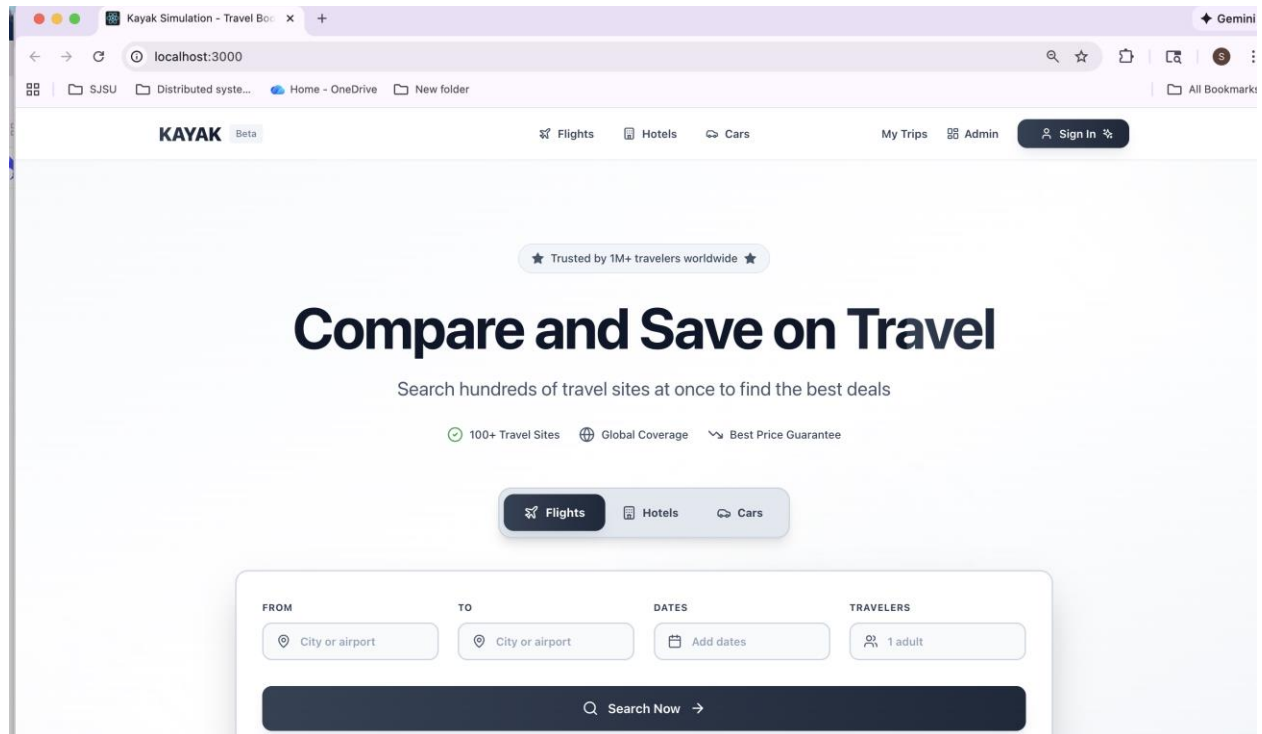
4. Verify Kafka Topics

docker-compose `exec` kafka kafka-topics --bootstrap-server localhost:9092 --list

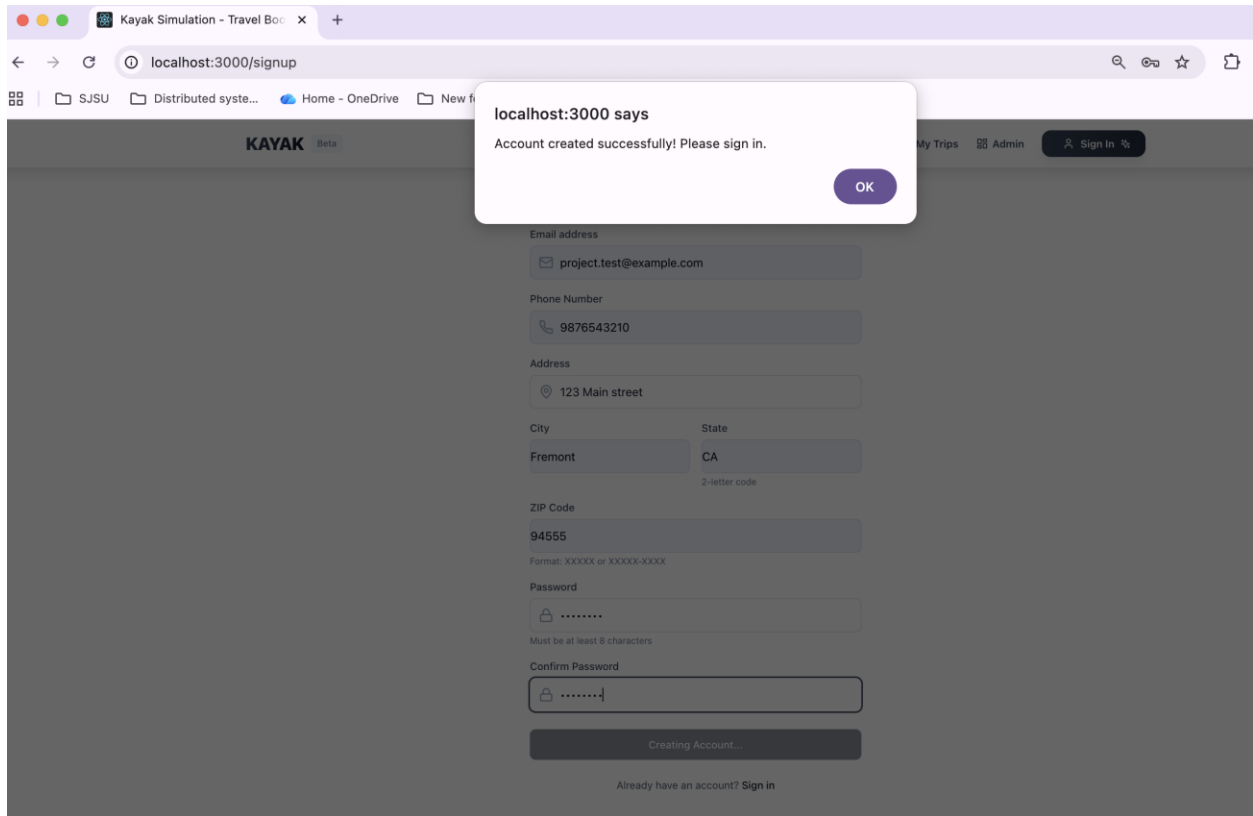
```
(base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % docker-compose exec kafka kafka-topics --bootstrap-server localhost:9092 --list
__consumer_offsets
kayak.ai.deal.events
kayak.ai.deals.normalized
kayak.ai.deals.scored
kayak.ai.deals.tagged
kayak.ai.raw_supplier_feeds
kayak.analytics.click
kayak.analytics.pageview
kayak.analytics.search
kayak.booking.cancelled
kayak.booking.completed
kayak.booking.created
kayak.booking.updated
kayak.listing.car.created
kayak.listing.car.updated
kayak.listing.flight.created
kayak.listing.flight.updated
kayak.listing.hotel.created
kayak.listing.hotel.updated
kayak.notification.email
kayak.notification.push
kayak.notification.sms
kayak.payment.completed
kayak.payment.failed
kayak.payment.initiated
kayak.payment.refunded
kayak.review.created
kayak.review.deleted
kayak.review.updated
kayak.search.car
kayak.search.flight
kayak.search.hotel
kayak.user.created
kayak.user.deleted
kayak.user.updated
```

5. Access Frontend

- Open browser: <http://localhost:3000>



Signup



Duplicate check

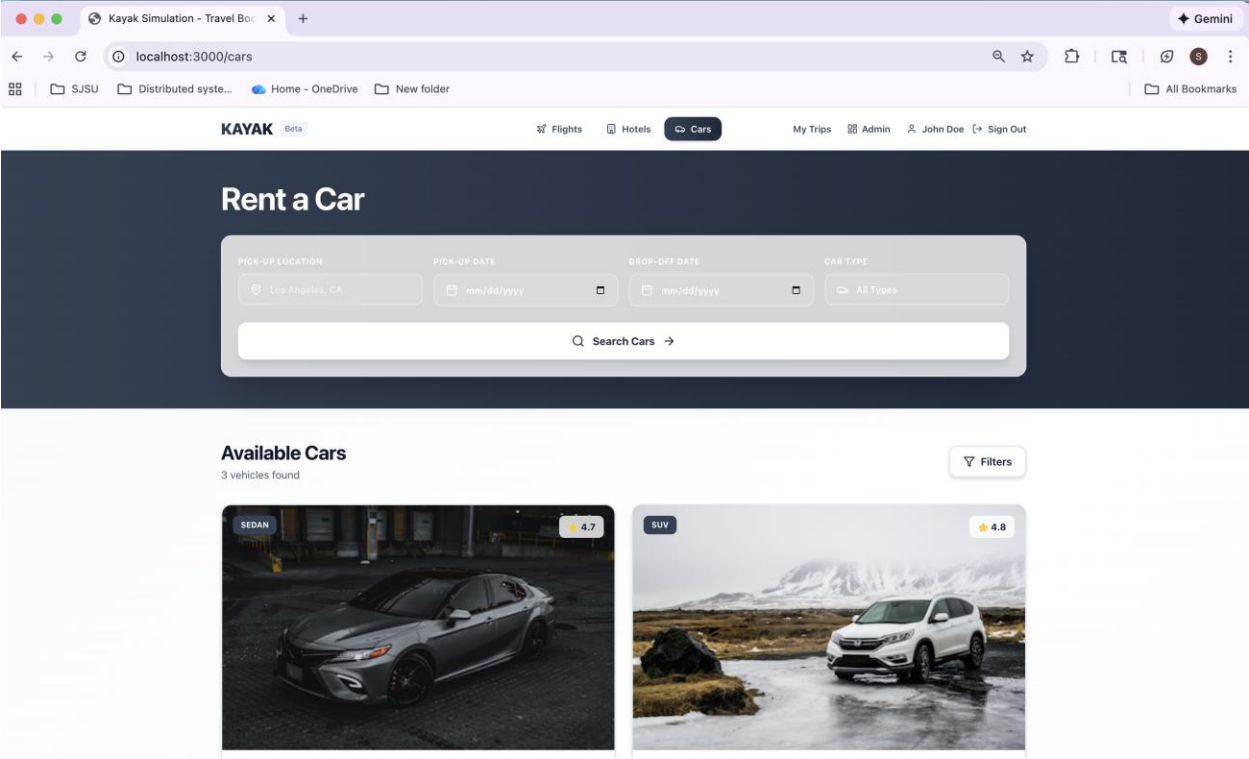
```
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl -X POST http://localhost:8001/users \
  -H "Content-Type: application/json" \
  -d '{
    "user_id": "123-45-6789",
    "first_name": "John",
    "last_name": "Doe",
    "email": "john.doe@example.com",
    "password": "SecurePass123!",
    "phone_number": "555-123-4567",
    "address": "123 Main Street",
    "city": "San Jose",
    "state": "CA",
    "zip_code": "95123"
  }'
{"detail":{"message":"User with ID 123-45-6789 already exists","error_code":"DUPLICATE_USER"}}
```

Test 1.2: User Registration - Invalid SSN Format

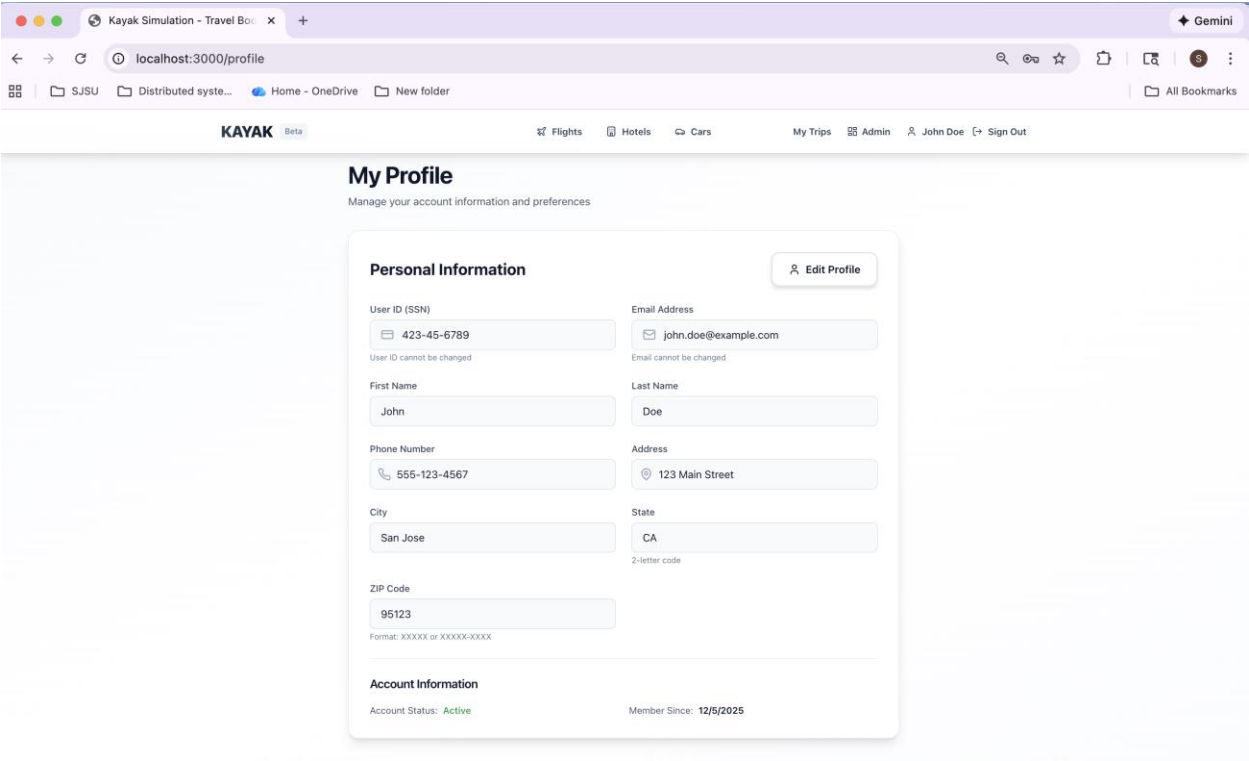
```
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl -X POST http://localhost:8001/users \
  -H "Content-Type: application/json" \
  -d '{
    "user_id": "423458789",
    "first_name": "John",
    "last_name": "Doe",
    "email": "john.doe@example.com",
    "password": "SecurePass123!",
    "phone_number": "555-123-4567",
    "address": "123 Main Street",
    "city": "San Jose",
    "state": "CA",
    "zip_code": "95123"
  }'
{"detail":{"message":"Invalid user ID format: 423458789. Required format: XXX-XX-XXXX","error_code":"INVALID_USER_ID","details":{"provided_id":"423458789"}}}
```

Test 1.3: User Login (JWT-based)

```
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl -X POST http://localhost:8001/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "john.doe@example.com",
    "password": "SecurePass123!"
  }'
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI0MjM0NTU0Njc4OSIsImV4cCI6MTc2NDkwNTY5Nn0.e0qoe4qf7ZHVFTF05HTBkSRHuF29Fv4QQMxAY2u73sI","token_type":"bearer","expires_in":1800,"user":{"user_id":"423-45-6789","first_name":"John","last_name":"Doe","email":"john.doe@example.com","phone_number":"555-123-4567","address":"123 Main Street","city":"San Jose","state":"CA","zip_code":"95123","profile_image_url":null,"credit_card_last_four":null,"credit_card_type":null,"is_active":true,"created_at":"2025-12-05T02:36:44","updated_at":"2025-12-05T02:36:44"}}}
```



Test 1.4: Profile Management



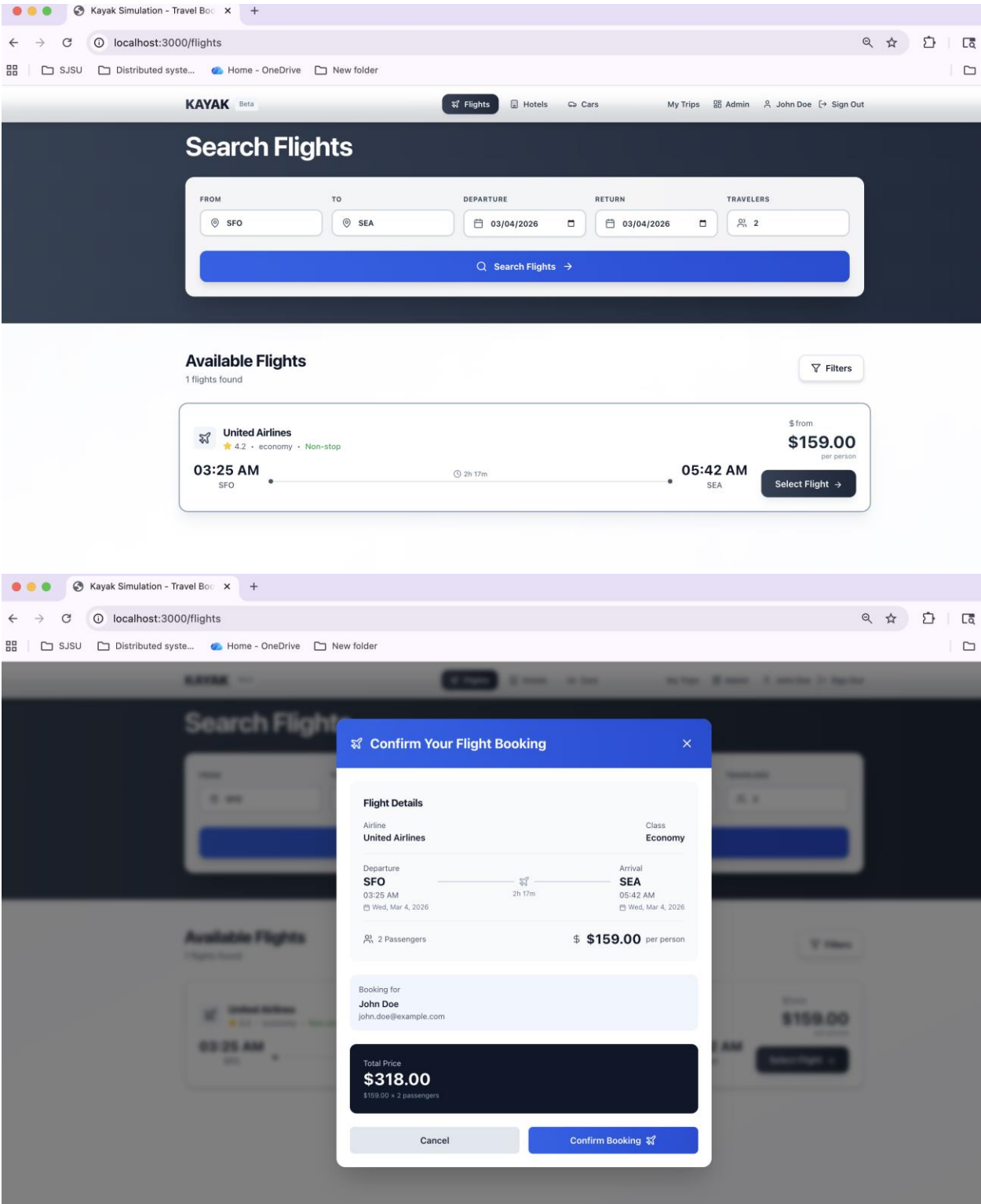
```
l": "2025-12-05T02:56:44"}]]%
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl -X PUT http://localhost:8001/users/123-45-6789 \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{
    "phone_number": "555-999-8888"
  }'
{"user_id": "123-45-6789", "first_name": "John Updated", "last_name": "User", "email": "testuser9@example.com", "phone_number": "555-999-8888", "address": "123 Test St", "city": "San Jose", "state": "CA", "zip_code": "95123", "profile_image_url": null, "credit_card_last_four": null, "credit_card_type": null, "is_active": true, "created_at": "2025-12-05T02:19:49", "updated_at": "2025-12-05T02:56:50"}
```

Test 1.5: View Past Bookings

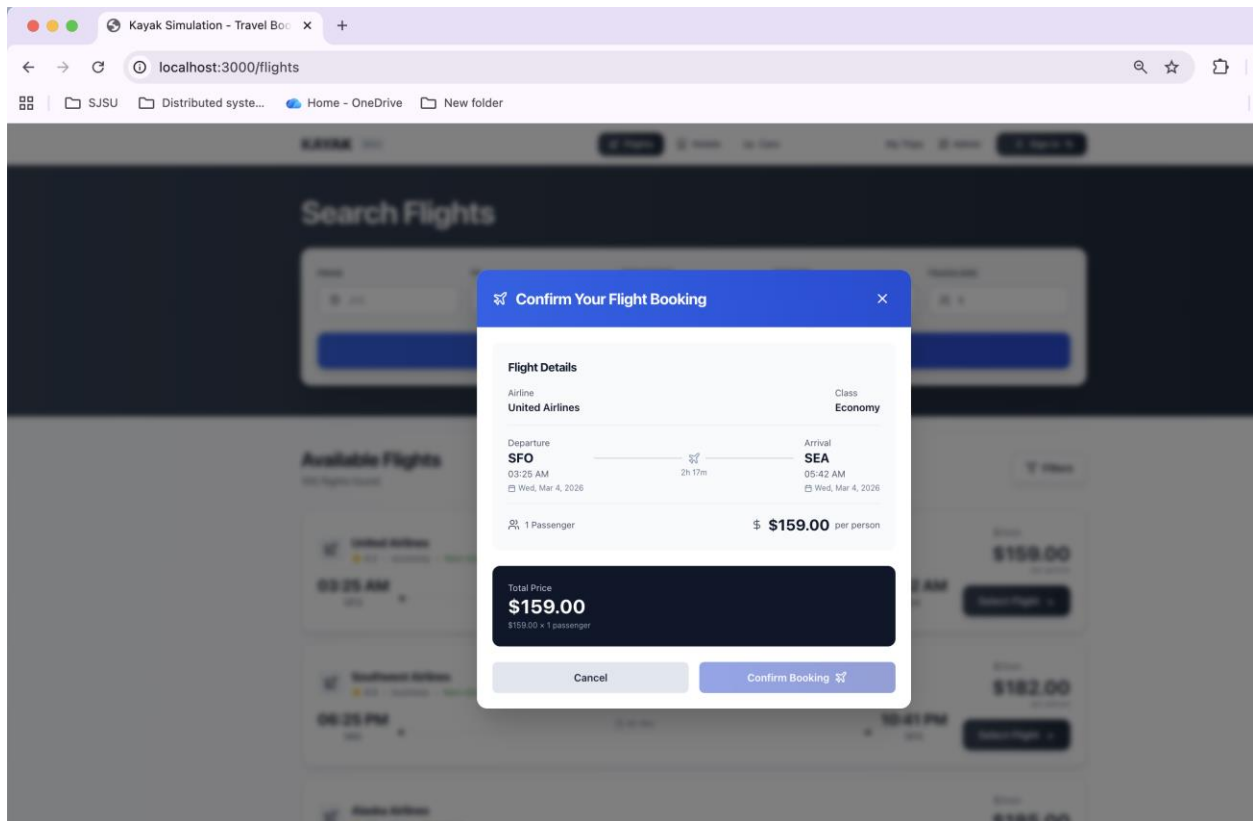
Test 1.6: View Upcoming Trips

Module 2: Search & Browsing Testing

Test 2.1: Search Flights



Unsigned bookings disabled



Pagination works

```
23h: Command not found: redis-cli
• (base) savithavijayarangan@Savithas-MacBook-Air: ~ % curl "http://localhost:8002/flights/search?departure_airport=SFO&arrival_airport=JFK&departure_date=2025-12-15&num_passengers=2&page=1&page_size=20"
{"flights": [], "total_count": 0, "page": 1, "page_size": 20, "total_pages": 1, "filters_applied": {"departure_airport": "SFO", "arrival_airport": "JFK", "departure_date": "2025-12-15", "num_passengers": 2, "sort_by": "price", "sort_order": "asc", "page": 1, "page_size": 20}}
```

Test 2.2: Search Hotels

localhost:3000/hotels

KAYAK Beta

Flights Hotels Cars My Trips Admin project test Sign Out

Find Your Perfect Stay

DESTINATION: New York

CHECK-IN: 12/11/2025

CHECK-OUT: 12/23/2025

GUESTS: 2

Search Hotels →

Available Hotels

11 of 11 properties found

Filters



4.4

Best Western New York

New York, NY

★★★★☆ (0 reviews)

Standard Room

wifi parking pool gym

\$ per night

\$150

Includes taxes & fees

View Details →



4.7

Hilton New York

localhost:3000/hotels

KAYAK Beta

Flights Hotels Cars My Trips Admin project test Sign Out

Find Your Perfect Stay

DESTINATION: New York

CHECK-IN: 12/11/2025

CHECK-OUT: 12/23/2025

GUESTS: 2

Search Hotels →

Available Hotels

11 of 11 properties found

Filters

Hotel Details



Best Western New York

8355 Main St, New York, NY 93824

★★★★☆ (4 stars) ★ 4.40

Description

Luxurious hotel in New York

Amenities

wifi parking pool gym

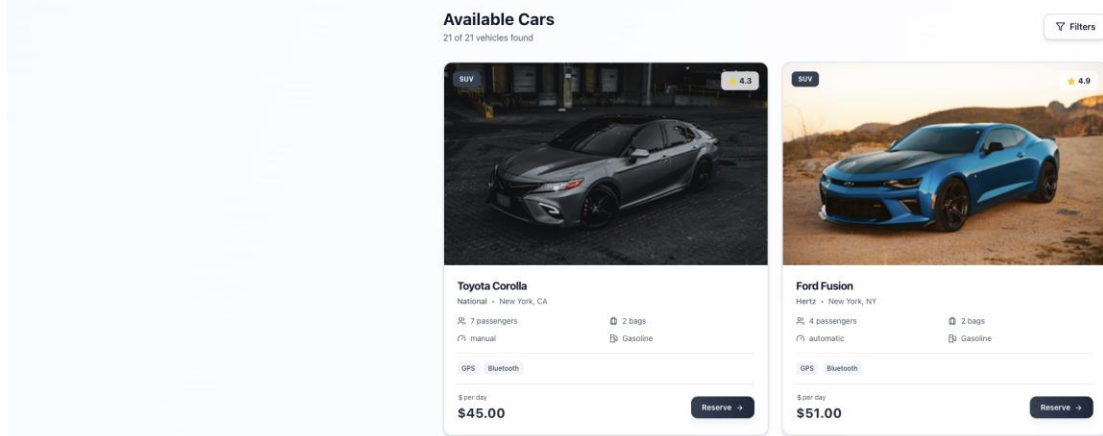
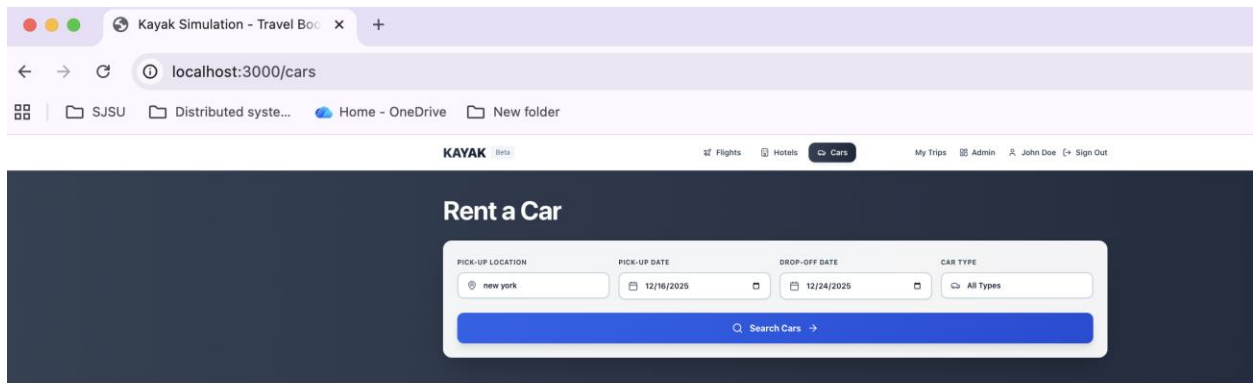
Close Book Now →

API Test:

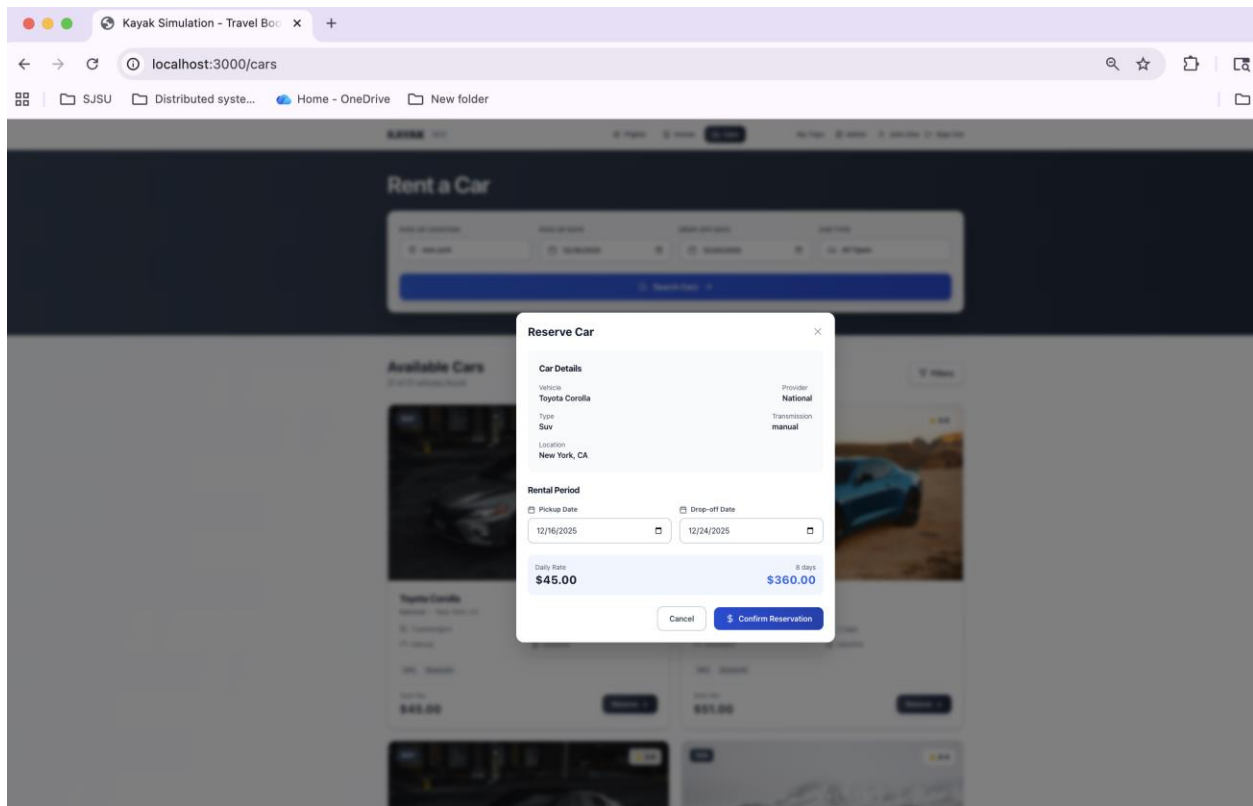
curl http://localhost:8003/hotels/search?city=San%20Francisco&check_in_date=2025-12-15&check_out_date=2025-12-20&num_guests=2

```
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl "http://localhost:8003/hotels/search?city=San%20Francisco&check_in_date=2025-12-15&check_out_date=2025-12-20&num_guests=2"
{"hotels":[{"hotel_id":"HTL-0020","hotel_name":"Best Western San Francisco","description":"Luxurious hotel in San Francisco","address":"3320 Main St","city":"San Francisco","state":"CA","zip_code":"84713","star_rating":4,"rating":4.4,"total_reviews":0,"amenities":"wifi,parking,pool,gym","phone_number":null,"email":null,"website":null,"is_active":true,"created_at":"2025-12-05T06:27:20","updated_at":"2025-12-05T06:27:20","rooms":[{"room_id":"HTL-0020-double-422","hotel_id":"HTL-0020","room_type":"double","room_number":null,"price_per_night":"90.00","max_occupancy":4,"total_rooms":6,"available_rooms":7,"is_active":true},{room_id":"HTL-0020-single-152","hotel_id":"HTL-0020","room_type":"single","room_number":null,"price_per_night":"107.00","max_occupancy":2,"total_rooms":14,"available_rooms":8,"is_active":true}]}],{"hotel_id":"HTL-0025","hotel_name":"Best Western San Francisco","description":"Luxurious hotel in San Francisco","address":"9783 Main St","city":"San Francisco","state":"CA","zip_code":"89885","star_rating":3,"rating":4.3,"total_reviews":0,"amenities":"wifi,parking,pool,gym","phone_number":null,"email":null}}
```

Test 2.3: Search Rental Cars



Filter

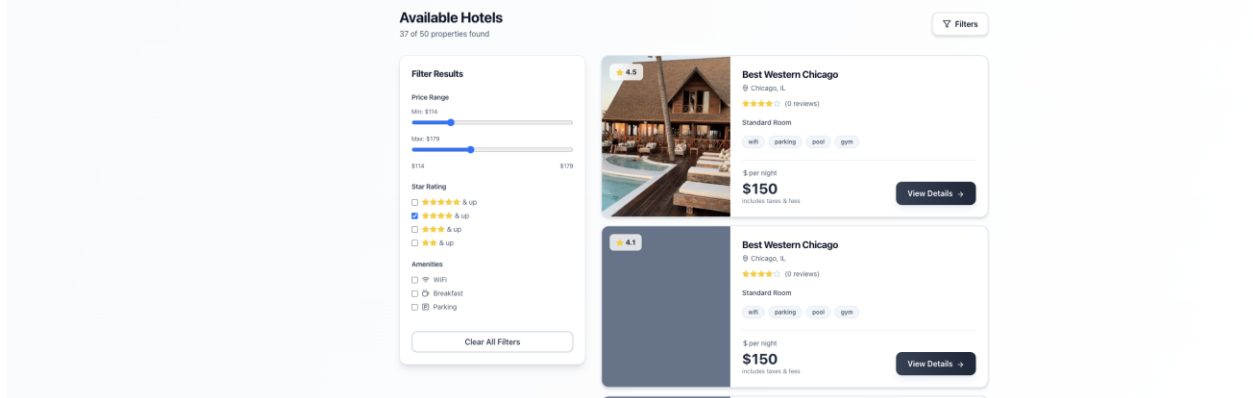
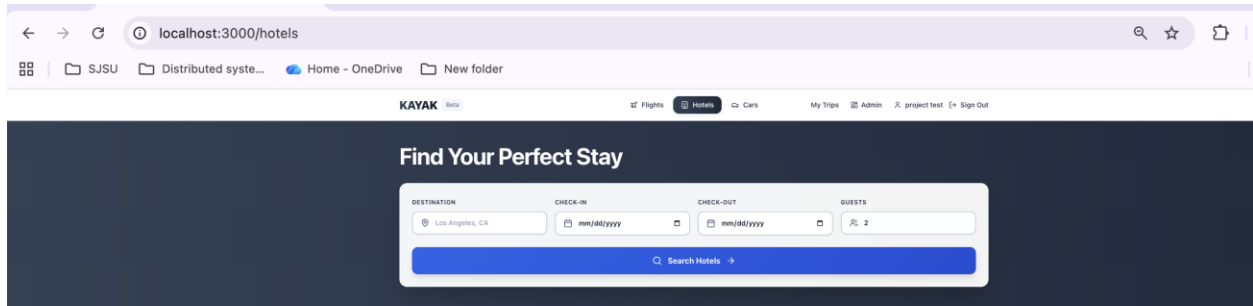
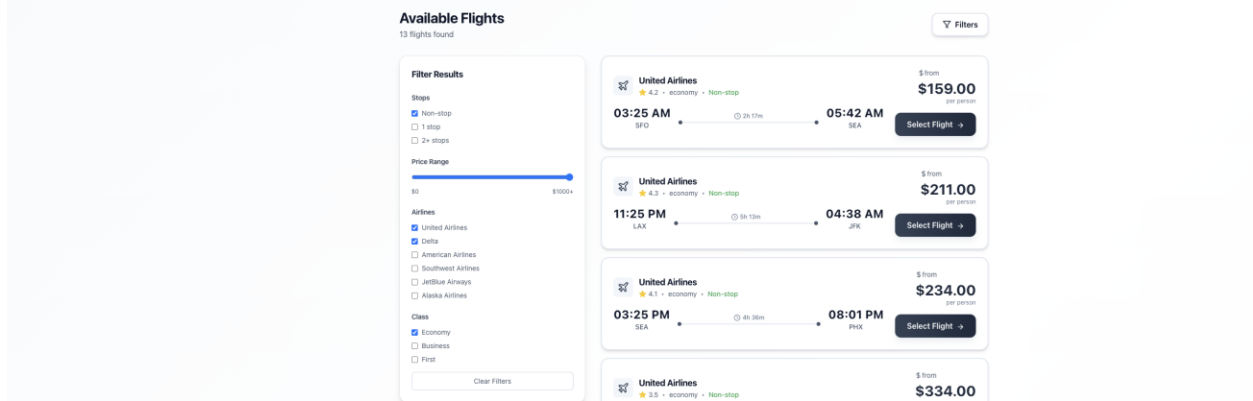
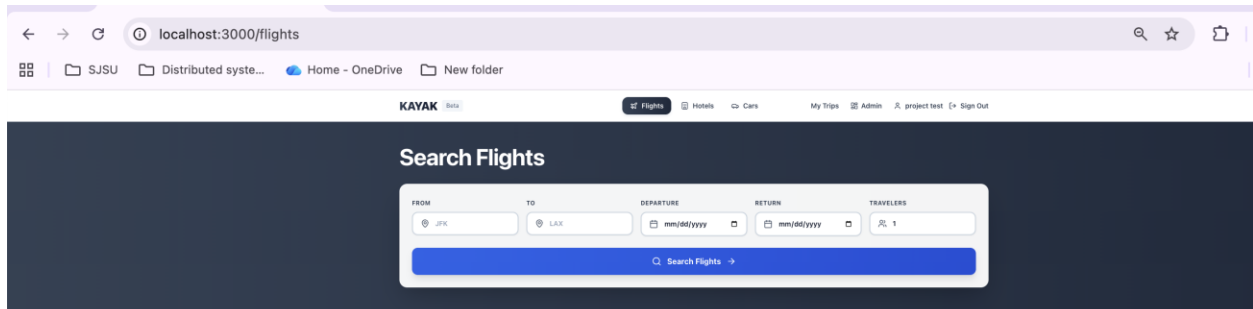


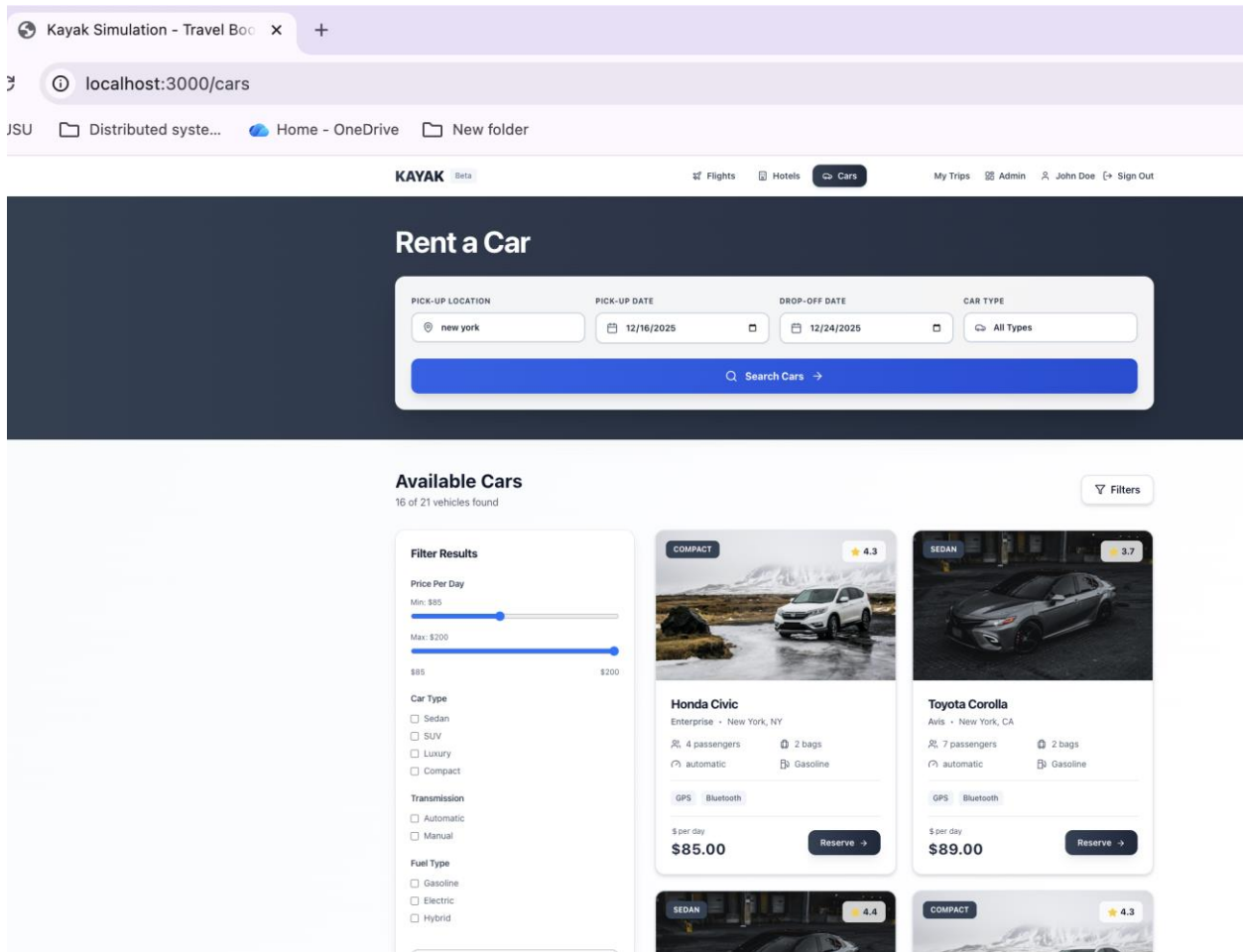
API test

curl <http://localhost:8004/cars/search?city=San%20Francisco>

```
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl "http://localhost:8004/cars/search?city=San%20Francisco"
{"cars":[{"car_id":"CAR-0062","car_type":"van","make":"Honda","model":"Civic","year":2024,"provider_name":"National","transmission_type":"manual","seats":4,"doors":4,"daily_rental_price":"47.00","pickup_location":"SFO Airport","city":"San Francisco","state":"CA","rating":4.0,"total_reviews":0,"is_available":true,"is_active":true,"created_at":"2025-12-05T06:27:20","updated_at":"2025-12-05T06:27:20"}, {"car_id":"CAR-0071","car_type":"luxury","make":"Toyota","model":"Corolla","year":2023,"provider_name":"Budget","transmission_type":"manual","seats":7,"doors":4,"daily_rental_price":"49.00","pickup_location":"JFK Airport","city":"San Francisco","state":"CA","rating":3.8,"total_reviews":0,"is_available":true,"is_active":true,"created_at":"2025-12-05T06:27:20","updated_at":"2025-12-05T06:27:20"}, {"car_id":"CAR-0041","car_
```

Test 2.4: Sort & Filter Results





curl

http://localhost:8002/flights/search?departure_airport=SFO&min_price=100&max_price=500&sort_by=price&sort_order=asc



Test 2.5: Unified Search Service

The screenshot shows the KAYAK search interface. The browser address bar displays `localhost:3000/search?city=San+Francisco`. The KAYAK logo and navigation links (Search, Flights, Hotels, Cars, My Trips, Admin, project test, Sign Out) are visible. The search form includes fields for City or Query, City (for hotels/cars), Departure Airport, Arrival Airport, Check-in Date, Check-out Date, Min Price, and Max Price. Below the form, the search results are displayed under the heading "Search Results" with a subtext "Found 61 total results". The results are categorized into three sections: Flights (20), Hotels (11), and Cars (20). Each section shows a grid of search results with details like route, airline, and price.

Search Results
Found 61 total results

Flights (20)

Route	Airline	Price
SFO → SEA	United Airlines	\$159.00
MIA → SFO	Southwest Airlines	\$182.00
SEA → ORD	Alaska Airlines	\$185.00
DFW → BOS	Southwest Airlines	\$208.00
LAX → JFK	United Airlines	\$211.00
ATL → BOS	United Airlines	\$212.00

Hotels (11)

Hotel Name	Location	Price
Best Western San Francisco	San Francisco, CA	\$4★
Best Western San Francisco	San Francisco, CA	\$3★
Hilton San Francisco	San Francisco, CA	\$3★
Hilton San Francisco	San Francisco, CA	\$5★
Holiday Inn San Francisco	San Francisco, CA	\$3★
Hyatt San Francisco	San Francisco, CA	\$5★

Cars (20)

Car Model	Location	Price
Honda Civic	San Francisco, CA	\$47.00/day
Toyota Corolla	San Francisco, CA	\$49.00/day
Honda Civic	San Francisco, CA	\$51.00/day

API Test:

curl "http://localhost:8007/search?city=San%20Francisco&search_type=hotel"

```
, page :1, page_size :20}}%
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl "ht
tp://localhost:8007/search?city=San%20Francisco&search_type=hotel"
{"flights": [], "hotels": [], "cars": [], "total_results": 0, "query": null, "filters
": {"query": null, "search_type": "hotel", "city": "San Francisco", "check_in": nul
l, "check_out": null, "departure_airport": null, "arrival_airport": null, "min_pri
ce": null, "max_price": null, "page": 1, "page_size": 20}}}%
○ (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation %
```

Test 3.1: Create Flight Booking

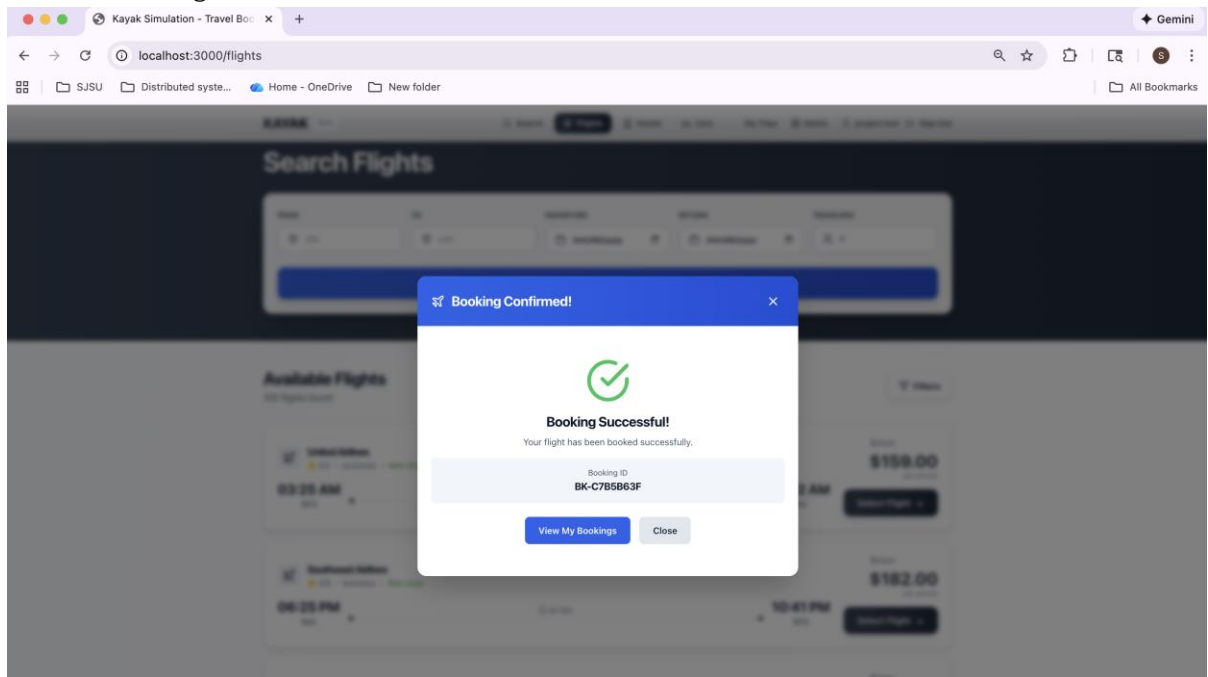
Objective: Verify flight booking creation

Prerequisites:

- User logged in
- Flight selected from search

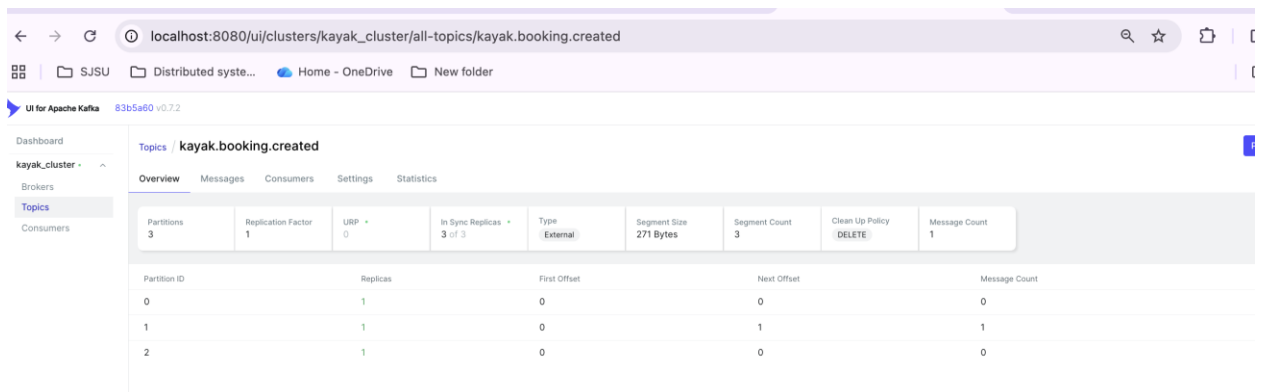
Steps:

1. Select a flight from search results
2. Click "Book Now" or "Select Flight"
3. Enter passenger details
4. Confirm booking



Expected Result:

- Booking created with status PENDING
- Available seats decremented
- Kafka event `booking.created` published



- Email notification triggered

Test 3.2: Create Hotel Booking

Objective: Verify hotel booking creation

Steps:

1. Select a hotel from search
2. Select room type
3. Enter check-in/check-out dates
4. Confirm booking

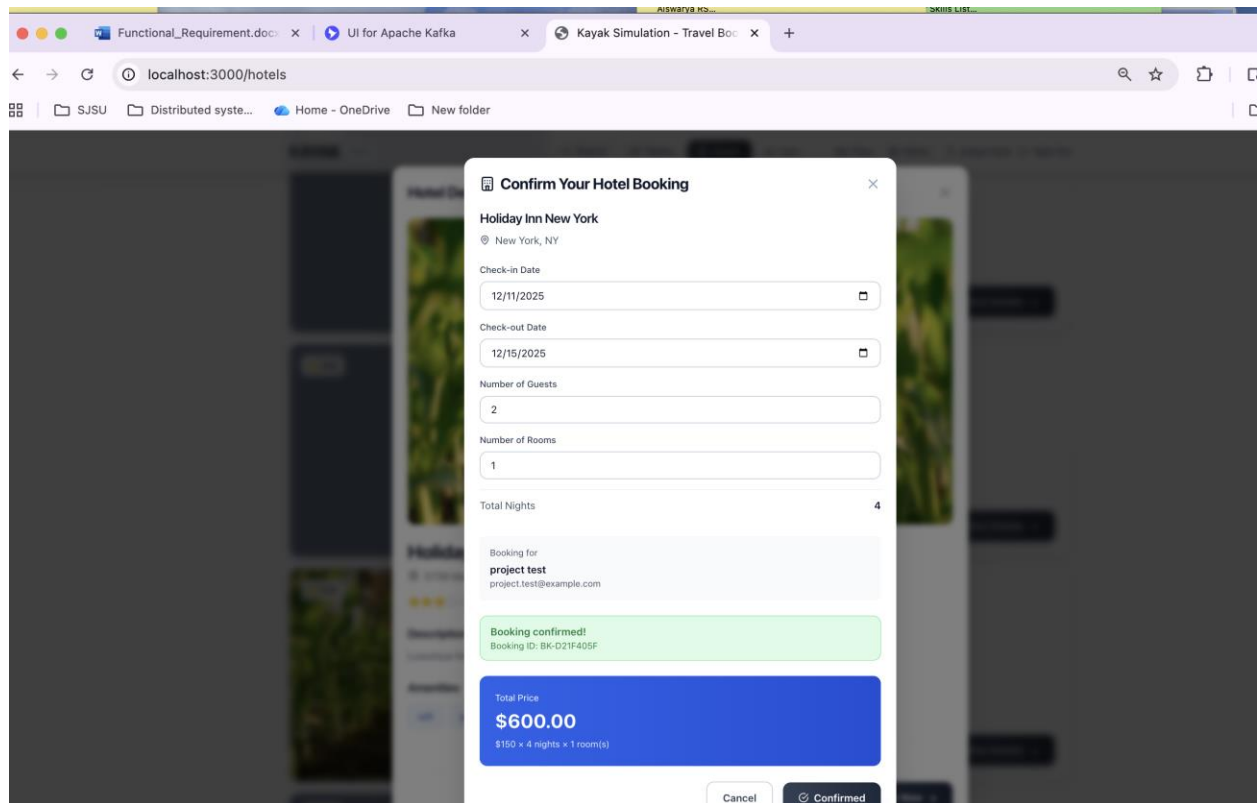
Expected Result:

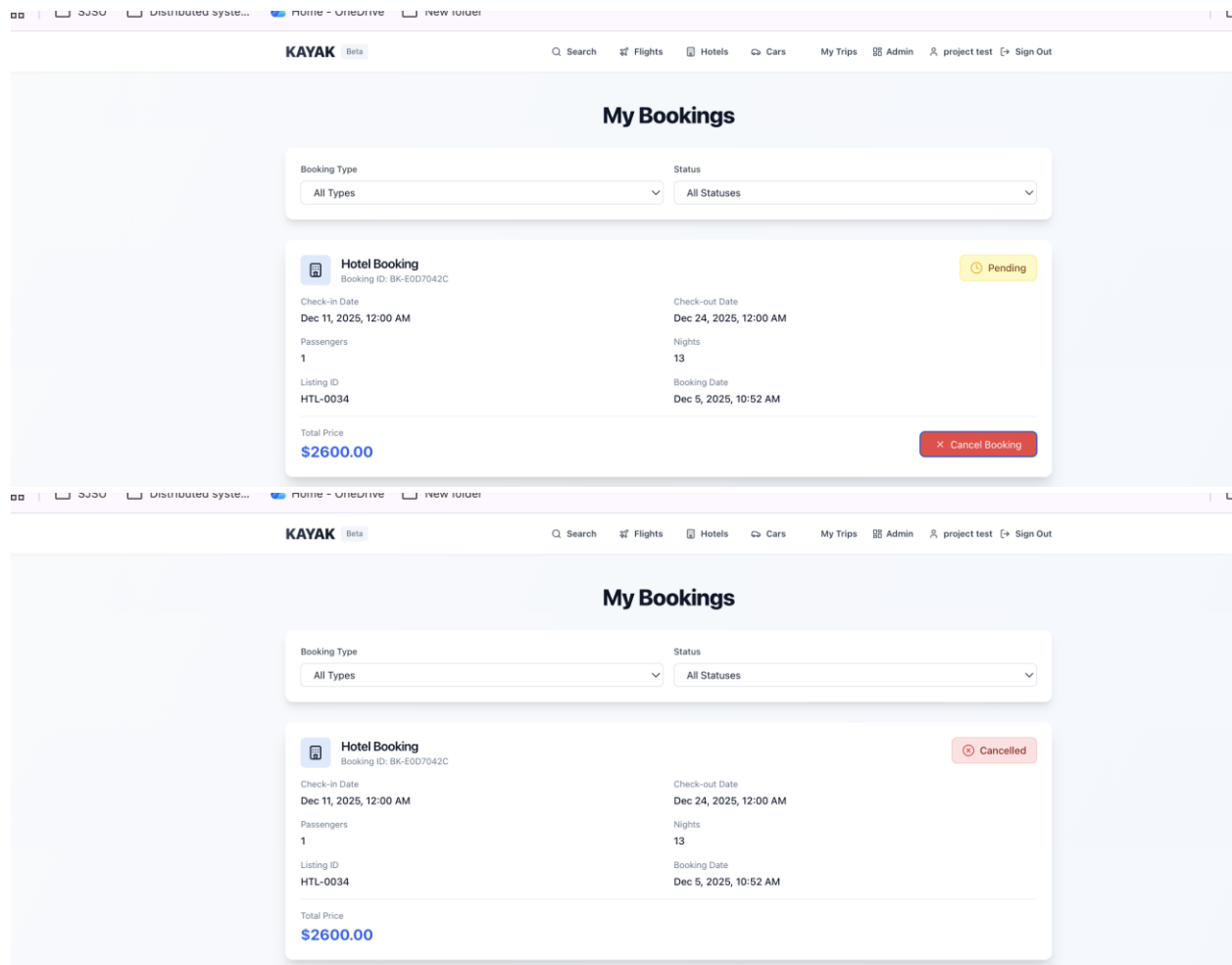
- Booking created
- Date overlap validation prevents double booking
- Room availability checked
- Price calculated (nights × price_per_night)

The screenshot shows a web browser window with a modal form titled "Confirm Your Hotel Booking" for "Best Western New York" in "New York, NY". The form contains the following fields and values:

- Check-in Date:** 12/05/2025
- Check-out Date:** 12/08/2025
- Number of Guests:** 3
- Number of Rooms:** 4
- Total Nights:** 3

Below the form fields, it says "Booking for project test" with the email "project.test@example.com". At the bottom, a blue box displays the "Total Price" as "\$900.00" with the calculation "\$150 × 3 nights × 2 room(s)". At the very bottom are "Cancel" and "Confirm Booking →" buttons.





Test 3.3: Create Car Booking

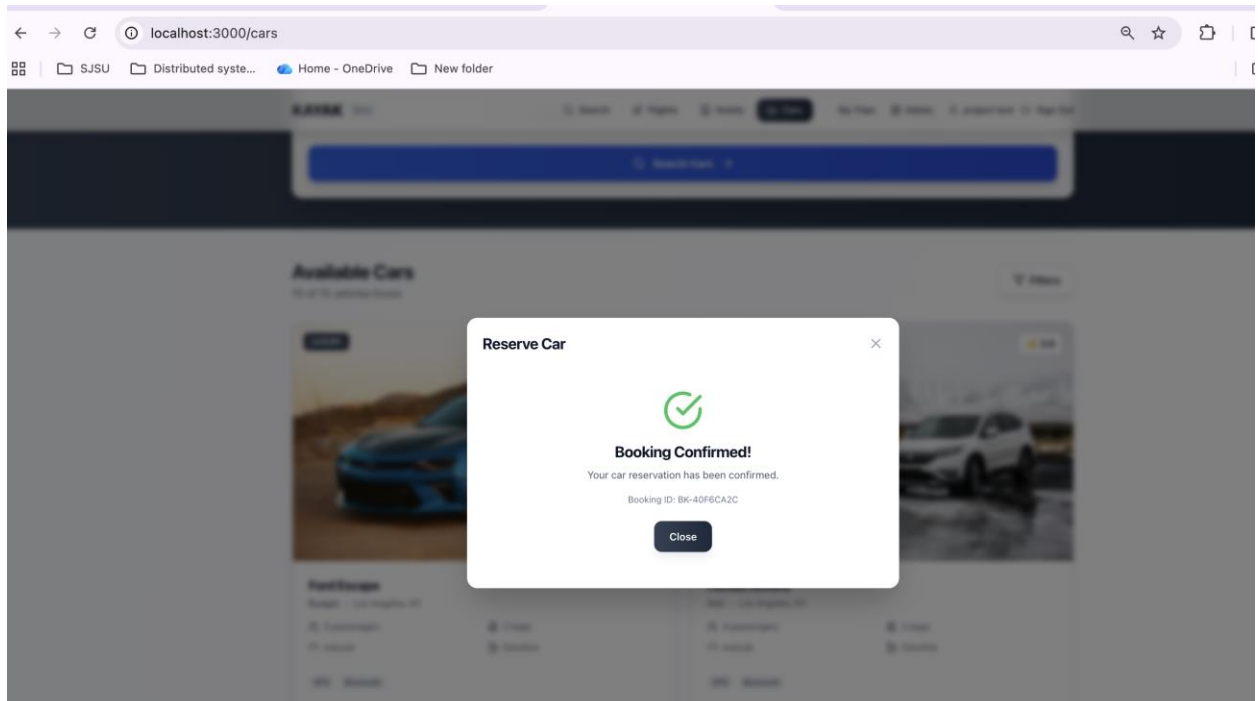
Objective: Verify car booking creation

Steps:

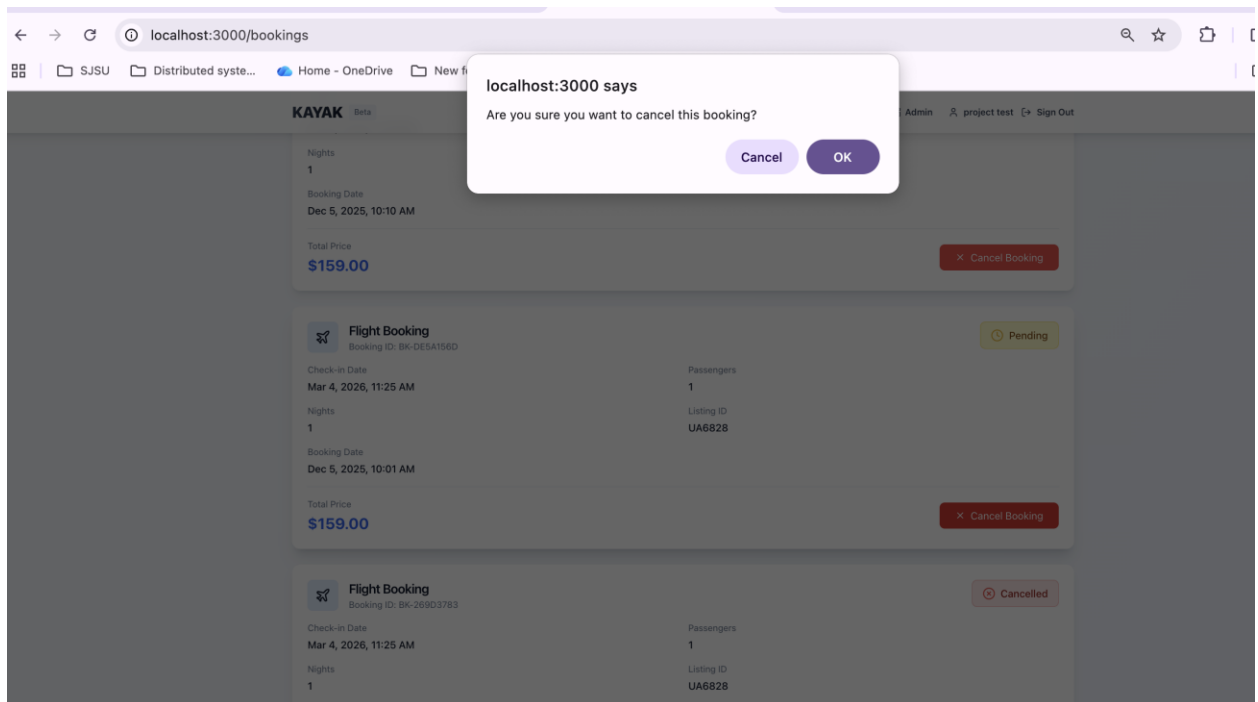
1. Select a car from search
2. Enter pickup/drop-off dates
3. Confirm booking

Expected Result:

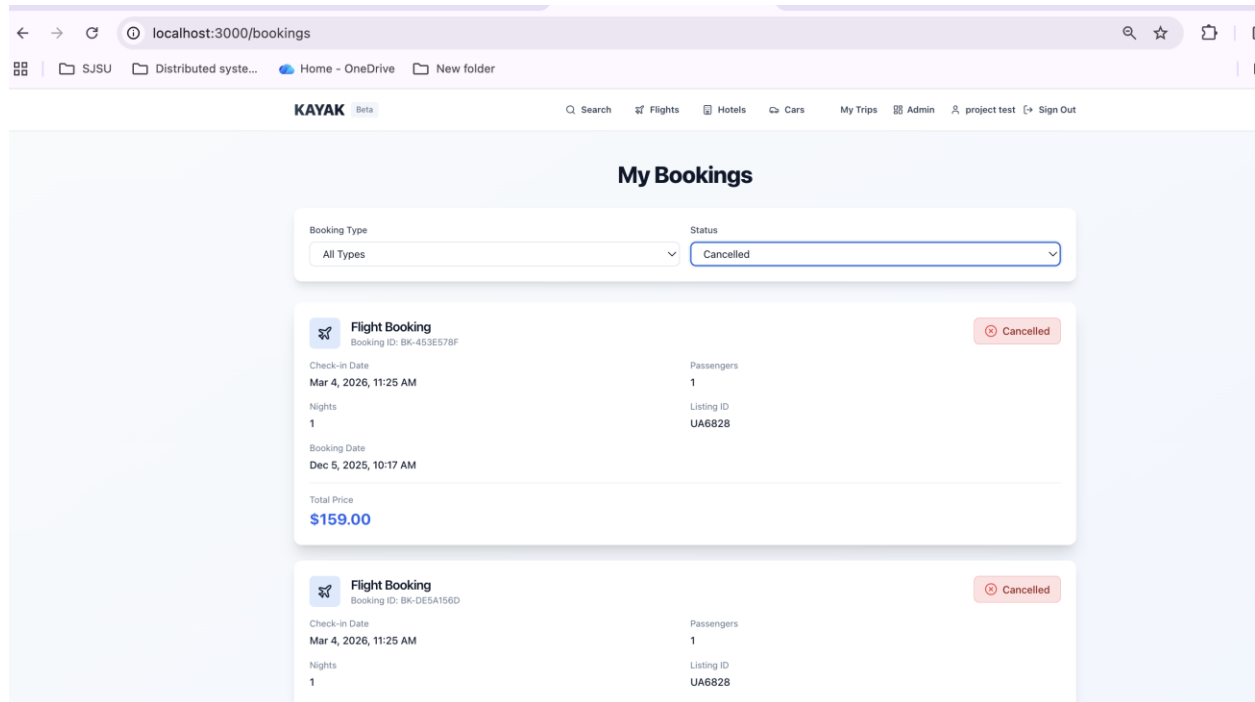
- Booking created
- Car availability checked
- Price calculated (days × price_per_day)



Cancelling from the user side



Filter based on status and type



Test 3.4: Prevent Double Booking (Date Overlap)

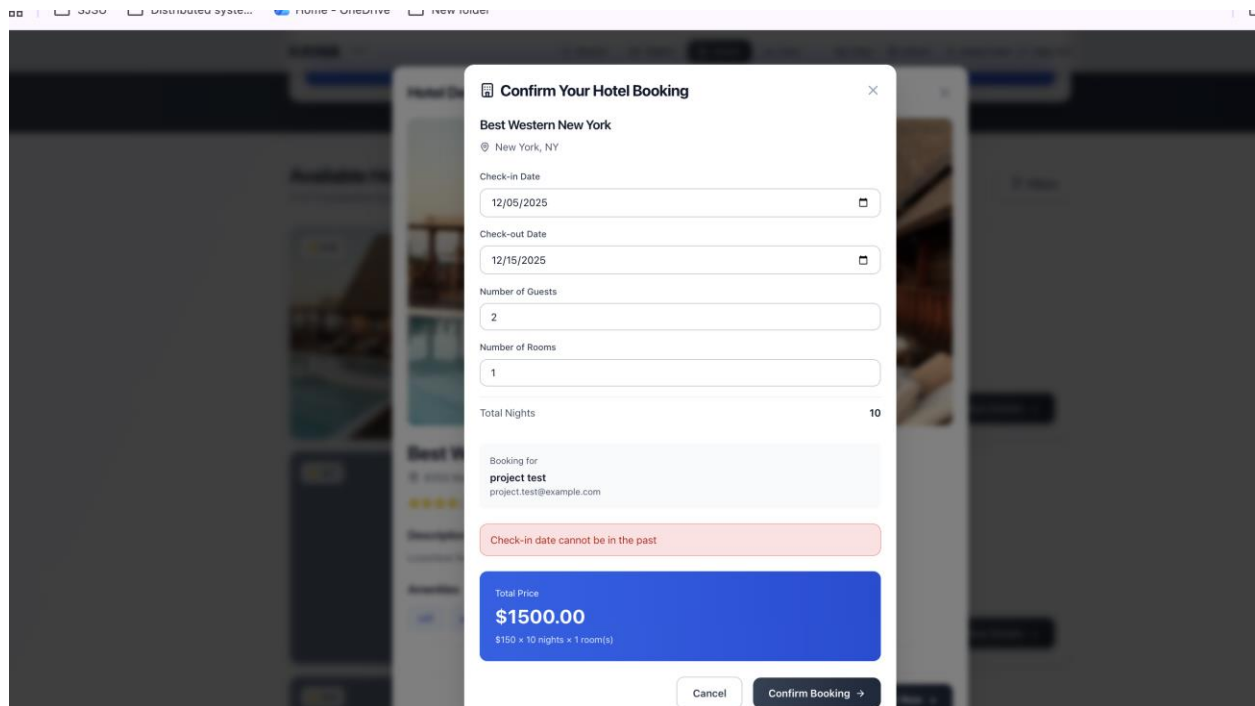
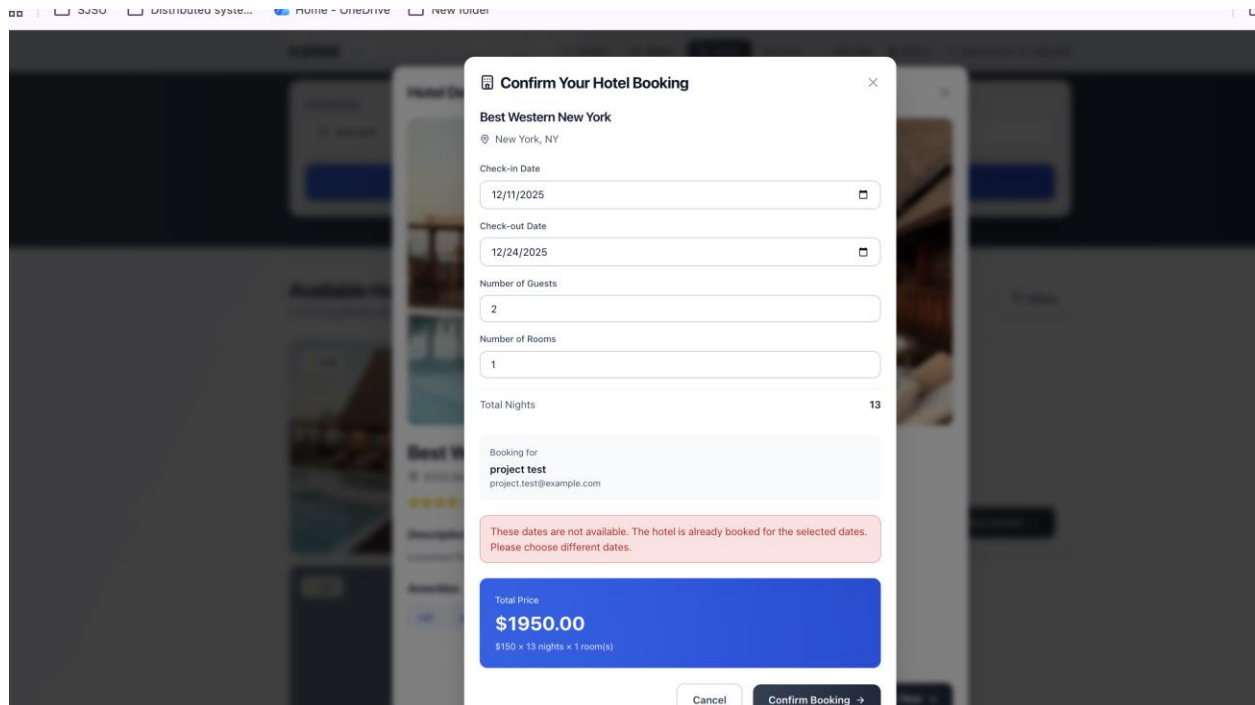
Objective: Verify system prevents overlapping bookings

Steps:

1. Create a hotel booking
2. Attempt to create another booking: (overlaps date)

Expected Result:

- Second booking rejected
- Error message: "Room not available for selected dates"
- First booking remains valid



Test 3.5: Check Availability

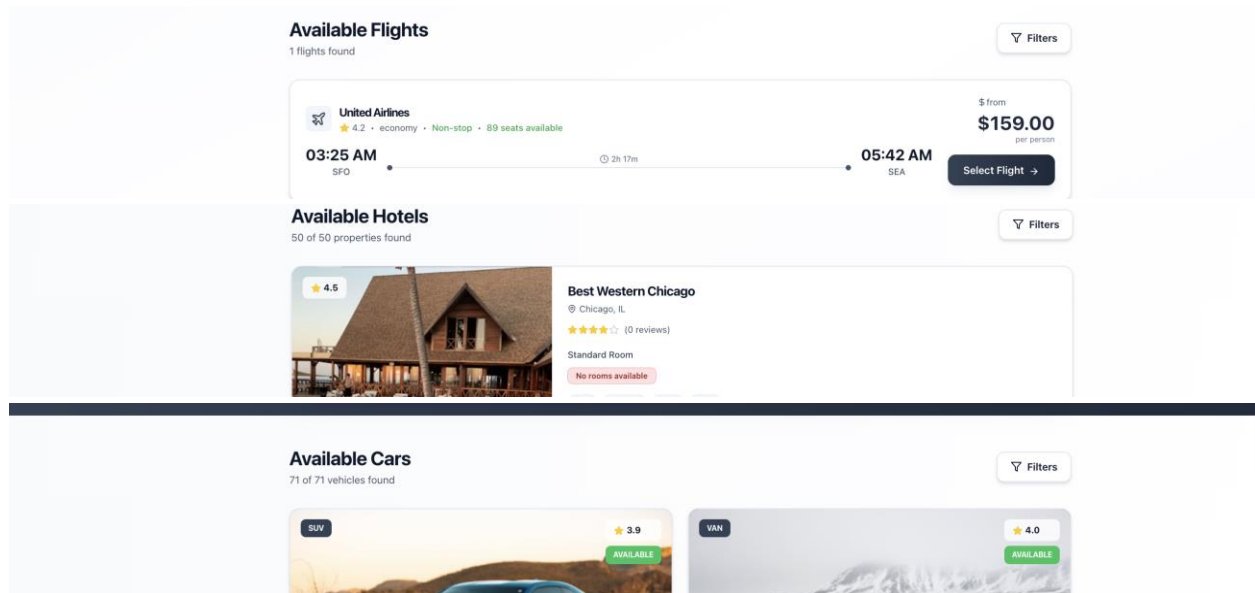
Objective: Verify availability checking works

Steps:

1. Search for flights/hotels/cars
2. Check availability for specific dates

Expected Result:

- Availability displayed correctly
- Real-time availability (seats/rooms)



Test 3.6: Price Calculation

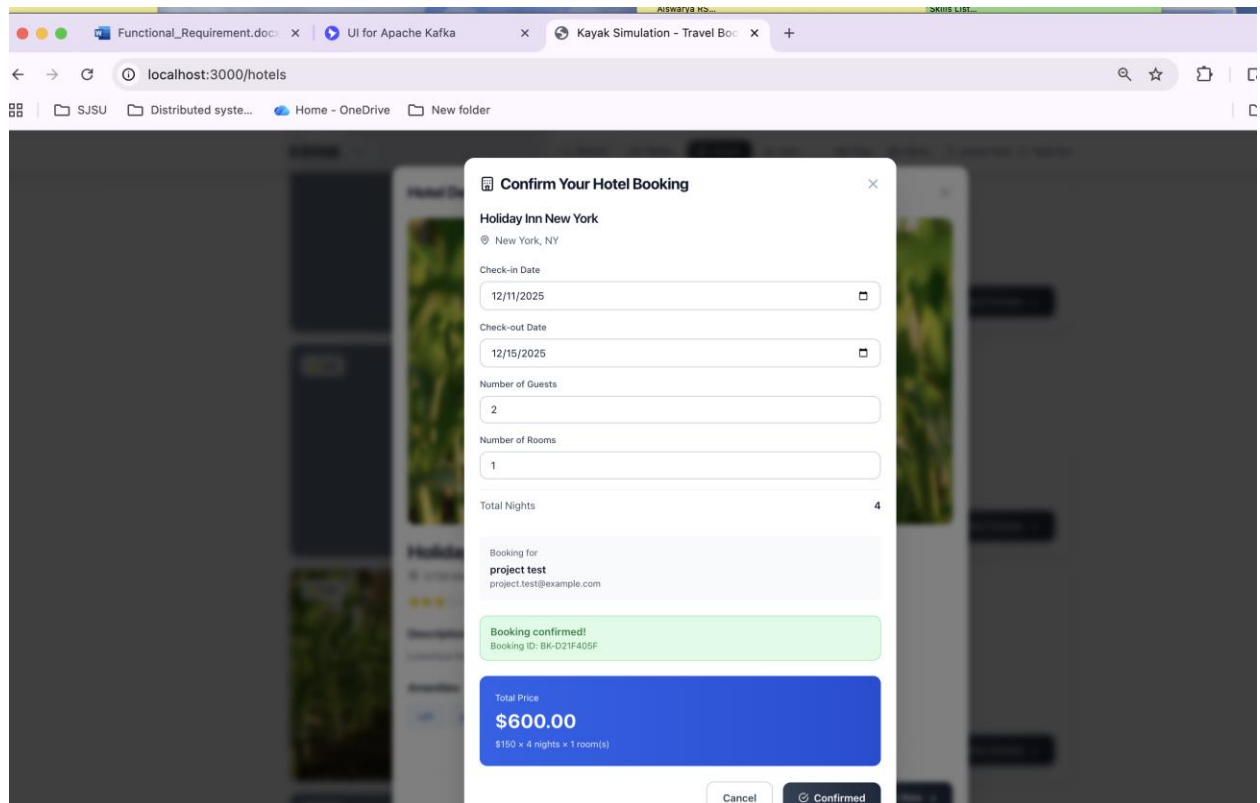
Objective: Verify price calculation is correct

Steps:

1. Create booking
2. Verify price calculation:
 - a. Flight: $\text{price} \times \text{passengers}$
 - b. Hotel: $\text{price_per_night} \times \text{nights} \times \text{rooms}$
 - c. Car: $\text{price_per_day} \times \text{days}$

Expected Result:

- Prices calculated correctly
- Tax included in billing



- Booking status changed to CANCELLED
- Inventory restored (seats/rooms)

- Refund processed (opt)
- Kafka event `booking.cancelled` published

Topics / `kayak.booking.cancelled`

Overview Messages Consumers Settings Statistics

Partitions 3	Replication Factor 1	URP 0	In Sync Replicas 3 of 3	Type External	Segment Size 1 KB	Segment Count 3	Clean Up Policy DELETE	Message Count 4
-----------------	-------------------------	----------	----------------------------	------------------	----------------------	--------------------	---------------------------	--------------------

Partition ID	Replicas	First Offset	Next Offset	Message Count
0	1	0	1	1
1	1	0	0	0
2	1	0	3	3

localhost:3000/bookings

KAYAK Beta

Search Flights Hotels Cars My Trips Admin project test Sign Out

Flight Booking

Booking ID: BK-02A94C32

Check-in Date

Mar 4, 2026, 11:25 AM

Nights

1

Booking Date

Dec 5, 2025, 10:10 AM

Total Price

\$159.00

Pending

Cancel Booking

Flight Booking

Booking ID: BK-DE5A156D

Check-in Date

Mar 4, 2026, 11:25 AM

Nights

1

Booking Date

Dec 5, 2025, 10:01 AM

Total Price

\$159.00

Cancelled

Cancelled

Flight Booking

Cancelled

Test 7.1: Payment Initiation

Objective: Verify payment processing

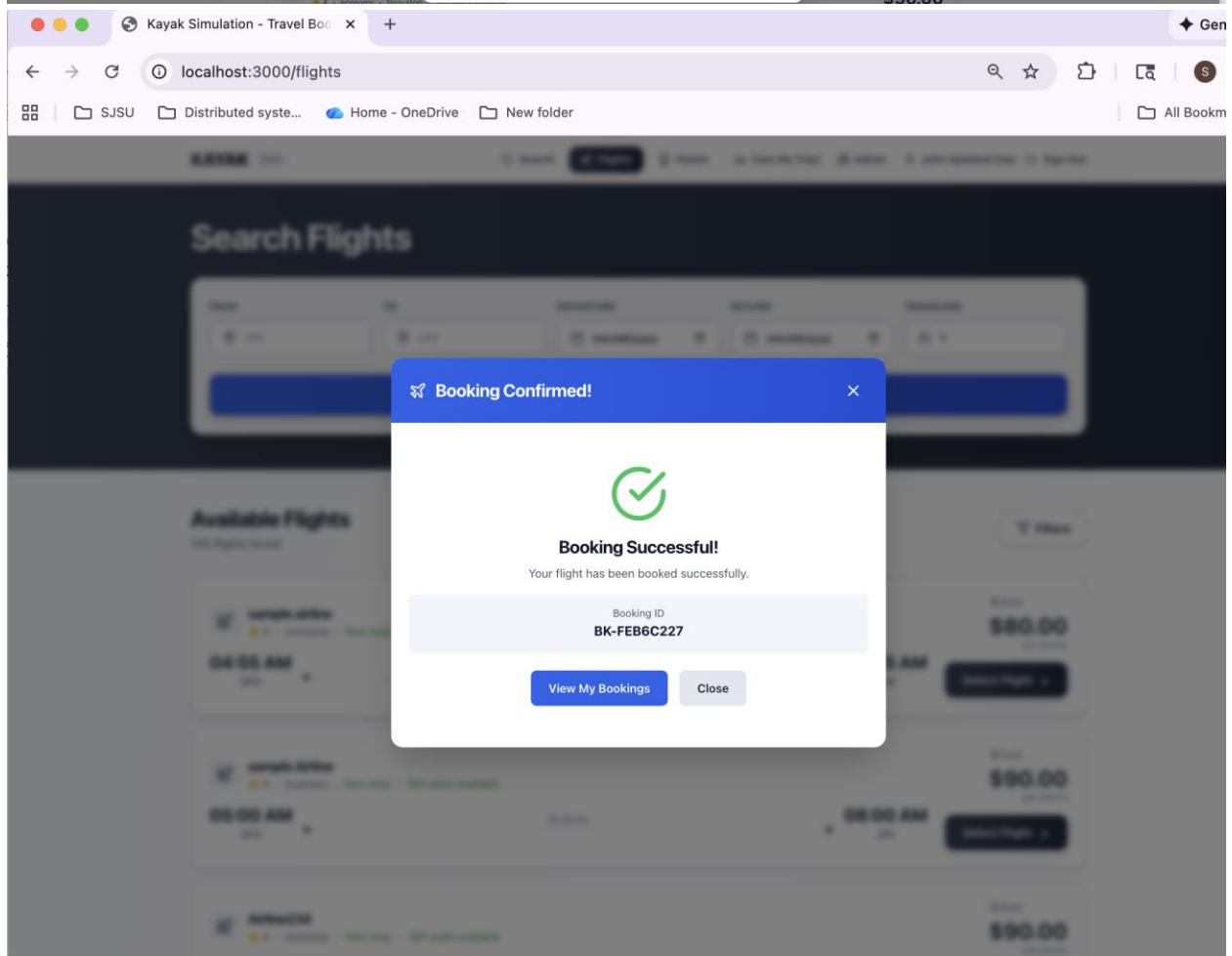
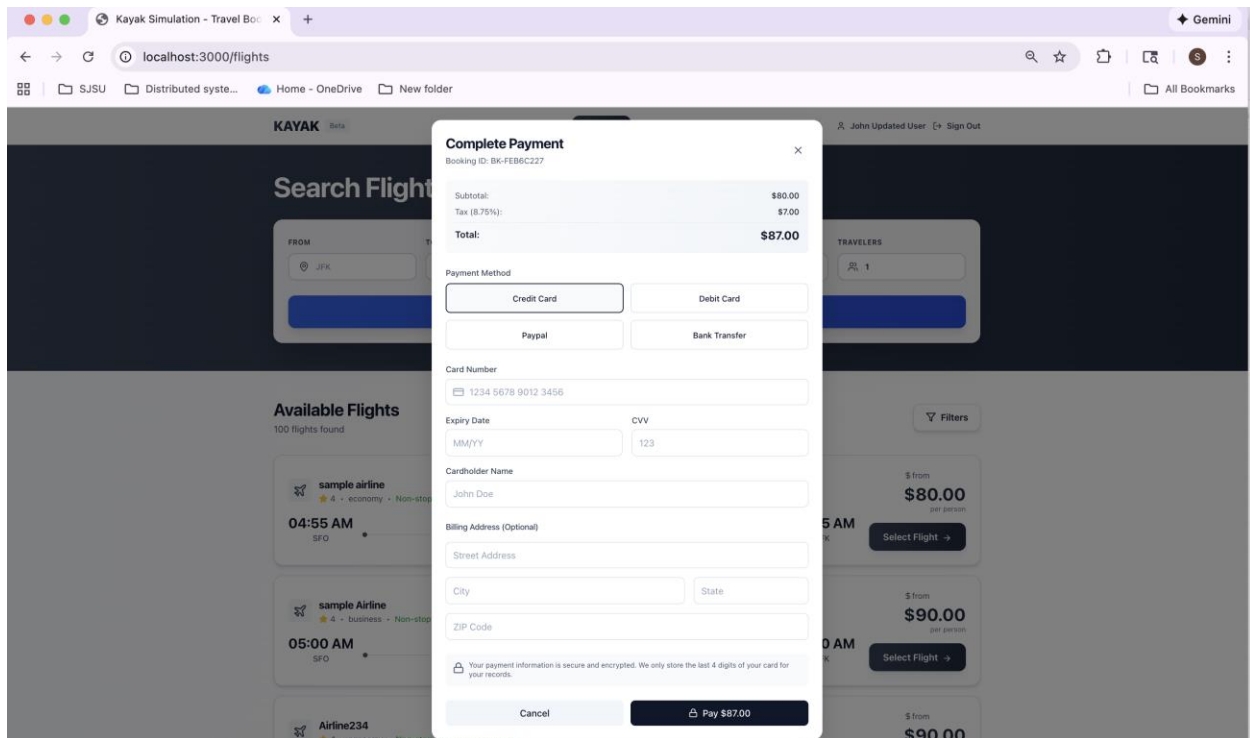
Steps:

1. Complete booking
2. Proceed to payment
3. Enter payment details:
 - a. Card: 4111111111111111
 - b. Expiry: 12/26
 - c. CVV: 123
 - d. Name: John Doe

4. Submit payment

Expected Result:

-  Payment processed
-  Invoice generated
-  Booking status: CONFIRMED
-  Kafka event `payment.completed` published



KAYAKBeta

SearchFlightsHotelsCarsMy TripsAdmin test adminSign Out

Admin Dashboard

Logout

Welcome back, test admin (admin)

Total Users

5

Total Revenue

\$348.00

Total Bookings

45

Active Listings

219

Manage Flights

View and manage flight listings

Manage Flights

Manage Hotels

View and manage hotel listings

Manage Hotels

Manage Cars

View and manage car listings

Manage Cars

Recent Bookings

View All →

Booking ID	Type	User ID	Status	Amount	Date
------------	------	---------	--------	--------	------

KAYAKBeta

SearchFlightsHotelsCarsMy TripsAdminJohn Updated UserSign Out

My Bookings

Booking Type

All Types

Status

All Statuses

Car Booking

Booking ID: BK-350ED874

Cancelled

Check-in Date

Dec 6, 2025, 12:00 AM

Check-out Date

Dec 7, 2025, 12:00 AM

Passengers

1

Nights

1

Listing ID

CAR-0044

Booking Date

Dec 5, 2025, 07:31 PM

Total Price

\$51.00

Flight Booking

Booking ID: BK-FEB6C227

Confirmed

Check-in Date

Dec 25, 2025, 12:55 PM

Check-out Date

Dec 26, 2025, 12:55 PM

Passengers

1

Nights

1

Listing ID

AA325

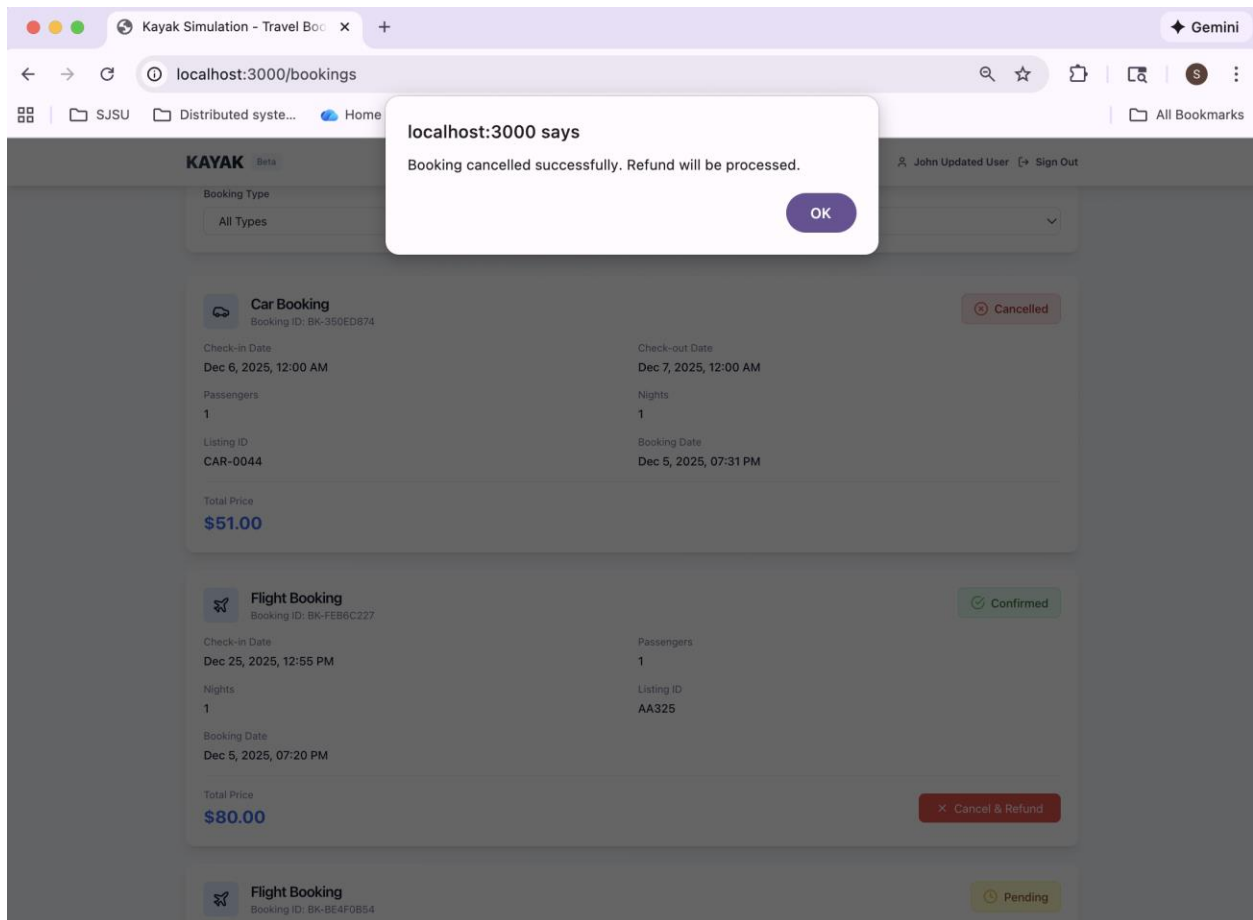
Booking Date

Dec 5, 2025, 07:20 PM

Total Price

\$80.00

Cancel & Refund



←

→

↺

localhost:3000/bookings

🔍

☆

🏠

🔍

📄

⌵

⋮

🗃️

SJSU

Distributed syste...

Home - OneDrive

New folder

All Bookmarks

KAYAKBeta

🔍 Search

✈️ Flights

🏨 Hotels

🚗 Cars My Trips

🗃️ Admin

👤 John Updated User

🔗 Sign Out

My Bookings

Booking Type

All Types

▼

Status

All Statuses

▼

✈️

Flight Booking

Booking ID: BK-0A347042

Check-in Date

Dec 25, 2025, 01:00 PM

Nights

1

Booking Date

Dec 5, 2025, 08:40 PM

Total Price

\$90.00

Passengers

1

Listing ID

AA765

Refund_pending

Refund Pending Admin Approval

Cancel from admin end

Bookings Management

Manage all customer bookings

Back to Dashboard

Booking Type

Status

All Types

All Statuses

Refresh

BOOKING ID	TYPE	USER ID	STATUS	AMOUNT	DATE	ACTIONS
BK-0A347042	Flight	123-45-6789	Refund_pending	\$90.00	Dec 5, 2025 at 08:40 PM	
BK-D8AA4442	Flight	123-45-6789	Confirmed	\$90.00	Dec 5, 2025 at 08:40 PM	

localhost:3000/admin/bookings

KAYAKBeta

My TripsAdmintest adminSign Out

Refund Management

Booking ID:
BK-0A347042

Billing ID:
BILL-06D1628F

Amount:
\$97.88

Payment Status:
Refund Pending Approval

Reason
Cancellation valid

Close

Reject

Approve

Dec 5, 2025 at 08:40 PM

Dec 5, 2025 at 08:38 PM

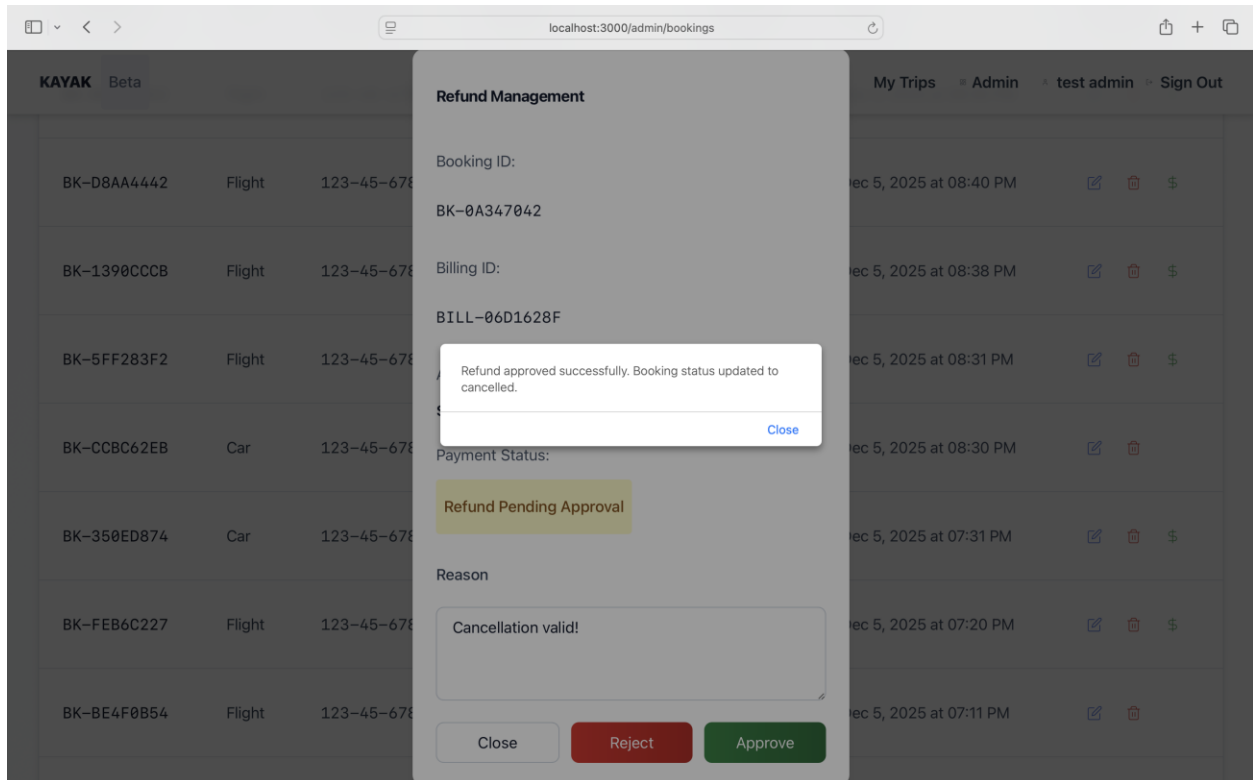
Dec 5, 2025 at 08:31 PM

Dec 5, 2025 at 08:30 PM

Dec 5, 2025 at 07:31 PM

Dec 5, 2025 at 07:20 PM

Dec 5, 2025 at 07:11 PM



Test 7.2: Tax Calculation (8.75%)

Objective: Verify tax calculation

Steps:

1. Process payment
2. Verify tax calculation

Expected Result:

- ☒ Tax = subtotal $\times$ 0.0875
- ☒ Total = subtotal + tax

Complete Payment

Booking ID: BK-0A347042

Subtotal:

\$90.00

Tax (8.75%):

\$7.87

Total:

\$97.88

Payment Method

Credit Card

Debit Card

Paypal

Bank Transfer

PayPal Email

project.test@example.com

Billing Address (Optional)

123 Main street

Fremont

CA

94555

 Your payment information is secure and encrypted. We only store the last 4 digits of your card for your records.

Cancel

 Pay \$97.88

Test 7.3: Invoice Generation

Objective: Verify invoice creation

Steps:

1. Complete payment
2. View invoice

Expected Result:

-  Invoice number generated

- ☒ Format: [INV-YYYYMMDD-XXXXXX](#)
- ☒ Invoice stored in database

Flight Booking
 Booking ID: BK-D8AA4442

Confirmed

Check-in Date
Dec 25, 2025, 01:00 PM

Passengers
1

Nights
1

Listing ID
AA543

Booking Date
Dec 5, 2025, 08:40 PM

Subtotal
 Tax (8.75%)

\$90.00
 \$7.88

Total
\$97.88

[Invoice: INV-20251205-C596B2](#)

Cancel & Request Refund

UI for Apache Kafka

83b5a60 v0.7.2

Dashboard

kayak_cluster

Brokers

Topics

Consumers

Topics

kayak.payment.completed

Produce Message

Overview

Messages

Consumers

Settings

Statistics

Partitions

3

Replication Factor

1

UIP

0

In Sync Replicas

3 of 3

Type

External

Segment Size

1 KB

Segment Count

3

Clean Up Policy

DELETE

Message Count

5

Partition ID

Replicas

First Offset

Next Offset

Message Count

0

1

0

2

2

1

1

0

1

1

2

1

0

2

2

Test 7.4: Payment Method Storage (Last 4 Digits)

Objective: Verify secure card storage

Steps:

1. Process payment with card
2. Check database

Expected Result:

- ☒ Only last 4 digits stored
- ☒ Full card number NOT stored
- ☒ Security compliance

```
mysql> SELECT
->     billing_id,
->     card_last_four,
->     LENGTH(card_last_four) as digit_count
-> FROM billings
-> WHERE card_last_four IS NOT NULL;
```

billing_id	card_last_four	digit_count
BILL-178778A8	1111	4
BILL-5B05AD97	1111	4
BILL-C88BEE83	1111	4
BILL-DDDC5C99	1111	4
BILL-FAA68426	1111	4

5 rows in set (0.00 sec)

Test 7.5: Payment Status Updates

Objective: Verify payment status tracking

Steps:

1. Process payment
2. Check payment status

Expected Result:

-  Status: COMPLETED
-  Booking status: CONFIRMED
-  Status history tracked



Flight Booking

Booking ID: BK-D8AA4442

Confirmed

Check-in Date

Dec 25, 2025, 01:00 PM

Nights

1

Booking Date

Dec 5, 2025, 08:40 PM

Passengers

1

Listing ID

AA543

Subtotal

\$90.00

Tax (8.75%)

\$7.88

Cancel & Request Refund

Total

\$97.88

Invoice: INV-20251205-C596B2

FLIGHTS

FLIGHTS

FLIGHTS

FLIGHTS

FLIGHTS

FLIGHTS

FLIGHTS

FLIGHTS

FLIGHTS

My Bookings

Booking Type

All Types

Status

All Statuses



Hotel Booking

Booking ID: BK-F1DCA0CE

Confirmed

Check-in Date

Dec 12, 2025, 12:00 AM

Check-out Date

Dec 16, 2025, 12:00 AM

Passengers

1

Nights

4

Listing ID

HTL-0001

Booking Date

Dec 5, 2025, 11:22 PM

Subtotal

\$372.00

Tax (8.75%)

\$32.55

Cancel & Request Refund

Total

\$404.55

Invoice: INV-20251205-149569 (View Details)

My Bookings

Booking Type

All Types

Status

All Statuses

 **Car Booking**
Booking ID: BK-7161A0CD

Confirmed

Check-in Date
Dec 18, 2025, 12:00 AM

Check-out Date
Dec 20, 2025, 12:00 AM

Passengers
1

Nights
1

Listing ID
CAR-0044

Booking Date
Dec 5, 2025, 10:20 PM

Subtotal
Tax (8.75%)

\$102.00
\$8.93

Cancel & Request Refund

Total

\$110.93

Invoice: INV-20251205-C3914C (View Details)

Test 7.6: Get Billing Record

Objective: Verify billing retrieval

Steps:

1. Retrieve billing by ID

Expected Result:

-  Billing details returned
-  All fields populated

← Back to Bookings

 Billing Details

Retrieve and view billing records by ID

Billing ID

BILL-20E297F6

🔍 Search

Billing Information

✔ Completed

Billing ID

BILL-20E297F6

Invoice Number

INV-20251205-C596B2

User ID

123-45-6789

Booking ID

BK-D8AA4442

Booking Type

Flight

Payment Method

Bank Transfer

Transaction Date

December 5, 2025 at 08:40 PM

Subtotal

\$90.00

Tax Amount (8.75%)

\$7.88

Total Amount

\$97.88

Created At

December 5, 2025 at 08:40 PM

Updated At

December 5, 2025 at 08:40 PM

📄 View Full Invoice

[← Back to Bookings](#)

Billing Details

Retrieve and view billing records by ID

Billing ID

BILL-1C7C83E9

Search

Billing Information

Completed

Billing ID

BILL-1C7C83E9

Invoice Number

INV-20251205-149569

User ID

123-45-6789

Booking ID

BK-F1DCA0CE

Booking Type

Hotel

Payment Method

Paypal

Transaction Date

December 5, 2025 at 11:22 PM

Subtotal

\$372.00

Tax Amount (8.75%)

\$32.55

Total Amount

\$404.55

Created At

December 5, 2025 at 11:22 PM

Updated At

December 5, 2025 at 11:22 PM

View Full Invoice

→ ↺ ⓘ localhost:3000/billing?booking_id=BK-7161A0CD 🔍 ☆ 📁

📁 SJSU 📁 Distributed syste... 🌐 Home - OneDrive 📁 New folder

KAYAK Beta 🔍 Search 🛫 Flights 🏠 Hotels 🚗 Cars My Trips ⚙️ Admin 👤 John Updated User ➔ Sign O

[← Back to Bookings](#)

💰 Billing Details

Retrieve and view billing records by ID

Billing ID

BILL-D5630F5B 🔍 Search

Billing Information

✔️ Completed

Billing ID

BILL-D5630F5B

Invoice Number

INV-20251205-C3914C

User ID

123-45-6789

Booking ID

BK-7161A0CD

Booking Type

Car

Payment Method

Bank Transfer

Transaction Date

December 5, 2025 at 10:20 PM

Subtotal

\$102.00

Tax Amount (8.75%)

\$8.93

Total Amount

\$110.93

Created At

December 5, 2025 at 10:20 PM

Updated At

December 5, 2025 at 10:20 PM

🔄 Refresh Invoice

Invoice Details

Invoice Number:

INV-20251205-C3914C

User Name:

John Updated User

User Email:

testuser9@example.com

Booking Type:

Car

Subtotal:

\$102.00

Tax Amount:

\$8.93

Total Amount:

\$110.93

Module 8: Email Notifications Testing

Test 8.1: Booking Confirmation Email

Objective: Verify booking email sent

Steps:

1. Create booking
2. Check email service logs

Expected Result:

- ☒ Email sent via Kafka event
- ☒ Email service consumed `booking.created` event
- ☒ Email content correct

Validation:

Check Kafka consumer

```
docker-compose logs email-service | grep "booking.created"
```

Check email service

```
docker-compose logs email-service | grep "Email sent"
```

```
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % docker-compose log
s email-service --since 2h 2>&1 | grep -E "EMAIL SENT|Sent booking confirmation"
kayak_email_service | INFO:backend.services.email_service.service: EMAIL SENT - To
: testuser9@example.com, Subject: Booking Confirmation - BK-7161A0CD
kayak_email_service | INFO:backend.services.email_service.consumer:Sent booking conf
irmation email for BK-7161A0CD
kayak_email_service | INFO:backend.services.email_service.service: EMAIL SENT - To
: testuser9@example.com, Subject: Booking Confirmation - BK-7FAFDAA
kayak_email_service | INFO:backend.services.email_service.consumer:Sent booking conf
irmation email for BK-7FAFDAA
kayak_email_service | INFO:backend.services.email_service.service: EMAIL SENT - To
: testuser9@example.com, Subject: Booking Confirmation - BK-F1DCA0CE
kayak_email_service | INFO:backend.services.email_service.consumer:Sent booking conf
irmation email for BK-F1DCA0CE
kayak_email_service | INFO:backend.services.email_service.service: EMAIL SENT - To
: testuser9@example.com, Subject: Booking Confirmation - BK-F3A2D606
kayak_email_service | INFO:backend.services.email_service.consumer:Sent booking conf
irmation email for BK-F3A2D606
kayak_email_service | INFO:backend.services.email_service.service: EMAIL SENT - To
: testuser9@example.com, Subject: Payment Confirmation - Invoice INV-20251205-5F6527
○ (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % `
┌ testuser9@example.com, Subject: Payment Confirmation - Invoice INV-20251205-5F6527
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % docker-compose exe
c kafka kafka-consumer-groups --bootstrap-server localhost:9092 --group email-service
-group --describe 2>/dev/null | grep "kayak.booking.created" | head -3

email-service-group kayak.booking.created 1 12 12
0 aiokafka-0.10.0-e0f4e4b4-47a3-496c-b238-9d8dac6563a1 /172.19.0.17
aiokafka-0.10.0
email-service-group kayak.booking.created 2 11 11
0 aiokafka-0.10.0-e0f4e4b4-47a3-496c-b238-9d8dac6563a1 /172.19.0.17
aiokafka-0.10.0
email-service-group kayak.booking.created 0 25 25
0 aiokafka-0.10.0-e0f4e4b4-47a3-496c-b238-9d8dac6563a1 /172.19.0.17
aiokafka-0.10.0
```

```

● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % docker-compose exe
c kafka kafka-topics --bootstrap-server localhost:9092 --list | grep booking
kayak.booking.cancelled
kayak.booking.completed
kayak.booking.created
kayak.booking.updated
● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation %

```

kayak_email_service

bcf565e5aafa kayak-simulation-email-service:latest

8010.8000

STATUS

Running (29 minutes ago)

Logs

Inspect

Bind mounts

Exec

Files

Stats

```

LIMIT %(param_1)s
INFO:sqlalchemy.engine.Engine:[generated in 0.00066s] {'user_id_1': '987-76-2421', 'param_1': 1}
INFO:backend.services.email_service.service: EMAIL SENT - To: project.test@example.com, Subject: Booking Confirmation - BK-C7B5B63F
INFO:backend.services.email_service.consumer:Sent booking confirmation email for BK-C7B5B63F
2025-12-05 10:20:18,519 INFO sqlalchemy.engine.Engine ROLLBACK
INFO:sqlalchemy.engine.Engine:ROLLBACK
INFO: 127.0.0.1:43458 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:50796 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:47550 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:42770 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:41124 - "GET /health HTTP/1.1" 200 OK

bookings.total_price AS bookings_total_price, bookings.booking_date AS bookings_booking_date, bookings.created_at AS bookings_created_at, bookings.updated_at AS bookings_updated_at
FROM bookings
WHERE bookings.booking_id = %(booking_id_1)s
LIMIT %(param_1)s
INFO:sqlalchemy.engine.Engine:[generated in 0.00082s] {'booking_id_1': 'BK-F3A2D606', 'param_1': 1}
2025-12-05 23:36:35,779 INFO sqlalchemy.engine.Engine [generated in 0.00082s] {'booking_id_1': 'BK-F3A2D606', 'param_1': 1}
INFO:backend.services.email_service.service: EMAIL SENT - To: testuser9@example.com, Subject: Payment Confirmation - Invoice INV-20251205-5F6527
INFO:backend.services.email_service.service: Template: payment_confirmation
INFO:backend.services.email_service.service: Context: {
  "billing_id": "BILL-86E568AA",
  "invoice_number": "INV-20251205-5F6527",
  "total_amount": 97.88,
  "payment_date": "2025-12-05T23:36:35",
  "booking_details": {
    "booking_id": "BK-F3A2D606",
    "booking_type": "Flight"
  }
}
INFO:backend.services.email_service.consumer:Sent payment confirmation email for billing BILL-86E568AA

```

EMAIL SENT

REG

1/49

Test 8.2: Payment Confirmation Email

Objective: Verify payment email sent

Steps:

1. Complete payment
2. Check email service logs

Expected Result:

- ☒ Email sent via Kafka event
- ☒ Email service consumed `payment.completed` event

Validation:

docker-compose logs email-service | grep "payment.completed"



kayak_email_service
bcf565e5aafa kayak-simulation-email-service:latest
8010:8000

STATUS
Running (29 minutes ago)

Logs Inspect Bind mounts Exec Files Stats

INFO:aiokafka.consumer.group_coordinator:Discovered coordinator 1 for group email-service-group
INFO:aiokafka.consumer.group_coordinator:Revoking previously assigned partitions set() for group email-service-group
INFO:aiokafka.consumer.group_coordinator:(Re-)joining group email-service-group
INFO:aiokafka.consumer.group_coordinator:Joined group 'email-service-group' (generation 1) with member_id aiokafka-0.10.0-313f0d30-f515-4d60-aac5-20f216a2a6f8
INFO:aiokafka.consumer.group_coordinator:Elected group leader -- performing partition assignments using roundrobin
INFO:aiokafka.consumer.group_coordinator:Successfully synced group email-service-group with generation 1
INFO:aiokafka.consumer.group_coordinator:Setting newly assigned partitions {TopicPartition(topic='kayak.booking.created', partition=1), TopicPartition(topic='kayak.payment.refunded', partition=1), TopicPartition(topic='kayak.payment.completed', partition=2), TopicPartition(topic='kayak.booking.created', partition=0), TopicPartition(topic='kayak.payment.completed', partition=1), TopicPartition(topic='kayak.payment.refunded', partition=0), TopicPartition(topic='kayak.booking.created', partition=2), TopicPartition(topic='kayak.payment.completed', partition=0), TopicPartition(topic='kayak.payment.refunded', partition=2)} for group email-service-group
INFO:backend.services.email_service.consumer:Email service consumer started
INFO: 127.0.0.1:42960 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:58424 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:52214 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:39752 - "GET /health HTTP/1.1" 200 OK

payment.completed x REG 2/58

Test 8.3: Email Service Consumer Running

Objective: Verify email consumer active

Steps:

1. Check Kafka consumer groups

Expected Result:

- ☒ email-service-group active
- ☒ Consuming from topics

Validation:

docker-compose exec kafka kafka-kafka-consumer-groups --bootstrap-server localhost:9092 --list

```
(base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % docker-compose exec kafka kafka-kafka-consumer-groups --bootstrap-server localhost:9092 --list
email-service-group
(base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation %
```

docker-compose exec kafka kafka-kafka-consumer-groups --bootstrap-server localhost:9092 --describe --group email-service-group

```

● (base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % docker-compose exec
c kafka kafka-consumer-groups --bootstrap-server localhost:9092 --describe --group em
ail-service-group

```

GROUP LAG CLIENT-ID	TOPIC CONSUMER-ID	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET HOST
email-service-group 0 aiokafka-0.10.0	kayak.booking.created aiokafka-0.10.0-e0f4e4b4-47a3-496c-b238-9d8dac6563a1	1	12	12 /172.19.0.17
email-service-group 0 aiokafka-0.10.0	kayak.payment.refunded aiokafka-0.10.0-e0f4e4b4-47a3-496c-b238-9d8dac6563a1	2	0	0 /172.19.0.17
email-service-group 0 aiokafka-0.10.0	kayak.payment.completed aiokafka-0.10.0-e0f4e4b4-47a3-496c-b238-9d8dac6563a1	2	3	3 /172.19.0.17
email-service-group 0 aiokafka-0.10.0	kayak.payment.completed aiokafka-0.10.0-e0f4e4b4-47a3-496c-b238-9d8dac6563a1	0	3	3 /172.19.0.17
email-service-group 0 aiokafka-0.10.0	kayak.payment.completed aiokafka-0.10.0-e0f4e4b4-47a3-496c-b238-9d8dac6563a1	1	2	2 /172.19.0.17

Partition #	Consumer ID #	Host #	Consumer Lag #	Current Offset #	End Offset #
2	aiokafka-0.10.0-e0f4e4b4-47a3-496c-b238-9d8dac6563a1	/172.19.0.17	0	11	11
1	aiokafka-0.10.0-e0f4e4b4-47a3-496c-b238-9d8dac6563a1	/172.19.0.17	0	12	12
0	aiokafka-0.10.0-e0f4e4b4-47a3-496c-b238-9d8dac6563a1	/172.19.0.17	0	25	25

Test 9.1: Admin Authentication

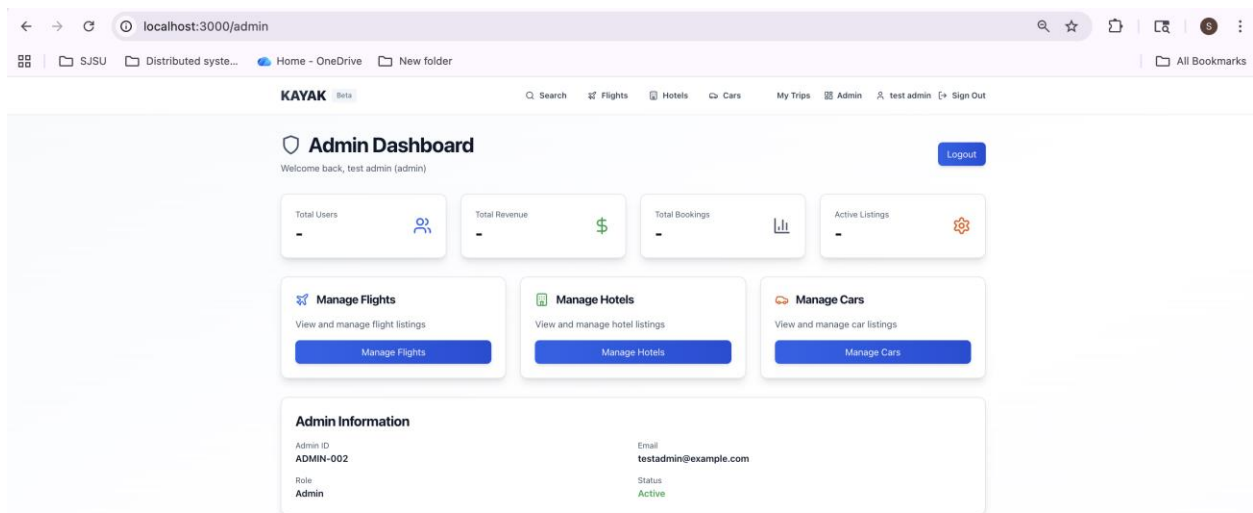
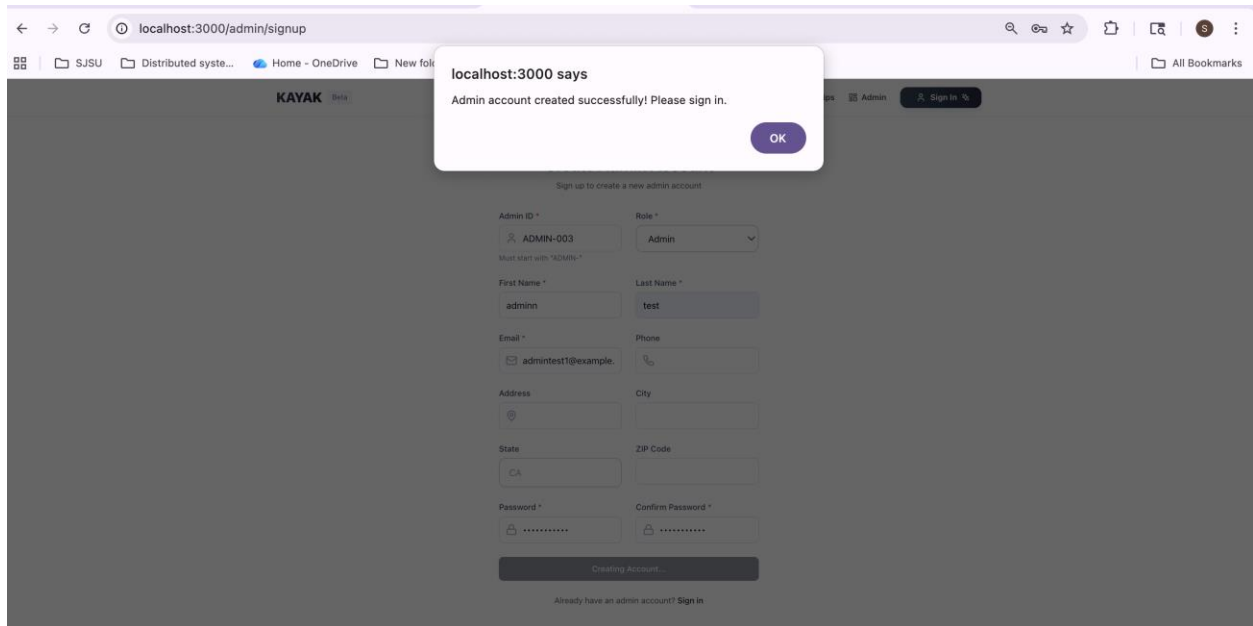
Objective: Verify admin login

Steps:

1. Login as admin
2. Verify admin dashboard access

Expected Result:

- ☒ Admin login successful
- ☒ Admin dashboard accessible
- ☒ Role-based access enforced



Test 9.2: Create/Update/Delete Flights

Objective: Verify flight management

Steps:

1. Admin creates flight
2. Admin updates flight
3. Admin deletes flight

Expected Result:

- All operations successful
- Changes reflected in search

localhost:3000/admin/flights

SJSU Distributed syste... Home - OneDrive New folder

KAYAK Beta

Search Flights Hotels Cars My Trips Admin test admin Sign Out

Back to Dashboard

Flight Management

Create, update, and delete flights

Create Flight

FLIGHT ID	AIRLINE	ROUTE	DEPARTURE	PRICE	SEATS	STATUS	ACTIONS
AA765	Airline234	SFO → JFK	Dec 25, 2025, 05:00 AM	\$90.00	180/180	Inactive	Edit Delete
AA543	sample Airline	SFO → JFK	Dec 25, 2025, 05:00 AM	\$90.00	199/200	Active	Edit Delete
AA325	sample airline	SFO → JFK	Dec 25, 2025, 04:55 AM	\$80.00	179/180	Active	Edit Delete
AA123	sample airline	SFO → JFK	Dec 25, 2025, 06:00 AM	\$100.00	200/200	Inactive	Edit Delete
AA999	American Airlines	SFO → JFK	Dec 15, 2025, 08:00 AM	\$349.99	180/180	Inactive	Edit Delete
AA1205	American Airlines	SEA → SFO	Dec 26, 2025, 04:25 AM	\$653.00	107/172	Active	Edit Delete
AA4089	American Airlines	BOS → DFW	Dec 25, 2025, 06:25 PM	\$615.00	97/194	Active	Edit Delete
AA5264	American Airlines	SFO → LAX	Feb 9, 2026, 02:25 AM	\$357.00	141/213	Active	Edit Delete
AA5574	American Airlines	LAS → LAX	Dec 7, 2025, 07:25 PM	\$486.00	110/192	Active	Edit Delete
AA7317	American Airlines	PHX → SEA	Feb 26, 2026, 05:25 PM	\$261.00	69/298	Active	Edit Delete
AA7825	American Airlines	PHX → ATL	Dec 29, 2025, 09:25 PM	\$721.00	105/179	Active	Edit Delete
AA8239	American Airlines	JFK → MIA	Feb 28, 2026, 12:25 AM	\$475.00	108/261	Active	Edit Delete
AA8715	American Airlines	BOS → DEN	Mar 4, 2026, 03:25 PM	\$334.00	61/240	Active	Edit Delete
AA8979	American Airlines	JFK → LAX	Feb 1, 2026, 05:25 PM	\$772.00	117/171	Active	Edit Delete

localhost:3000/admin/hotels

SJSU Distributed syste... Home - OneDrive New folder

KAYAK Beta

Search Flights Hotels Cars My Trips Admin test admin Sign Out

Back to Dashboard

Hotel Management

Create, update, and delete hotels

Create Hotel

HOTEL ID	NAME	LOCATION	STARS	STATUS	ACTIONS
HOTEL-098	Hotel98	Fremont, CA	★★★★★	Active	Edit Delete
HOTEL-005	hotel sample	Fremont, CA	★★★★	Active	Edit Delete
HTL-0026	Marriott San Francisco	San Francisco, CA	★★★★	Active	Edit Delete
HTL-0050	Holiday Inn San Francisco	San Francisco, CA	★★★★	Active	Edit Delete
HTL-0027	Marriott New York	New York, NY	★★★★	Active	Edit Delete
HTL-0028	Best Western Chicago	Chicago, IL	★★★★	Active	Edit Delete
HTL-0029	Holiday Inn New York	New York, NY	★★★★	Active	Edit Delete
HTL-0030	Hilton Miami	Miami, FL	★★★★	Active	Edit Delete
HTL-0031	Best Western Miami	Miami, FL	★★★★	Active	Edit Delete
HTL-0032	Best Western Miami	Miami, FL	★★★★★	Active	Edit Delete
HTL-0033	Holiday Inn Los Angeles	Los Angeles, CA	★★★★	Active	Edit Delete
HTL-0034	Best Western New York	New York, NY	★★★★	Active	Edit Delete
HTL-0035	Hilton Chicago	Chicago, IL	★★★★★	Active	Edit Delete
HTL-0036	Best Western Miami	Miami, FL	★★★★	Active	Edit Delete

localhost:3000/admin/hotels

SJSU Distributed syste... Home - OneDrive New folder

All Bookmarks

KAYAKBeta

SearchFlightsHotelsCarsMy TripsAdmintest adminSign Out

Back to Dashboard

Hotel Management

Create, update, and delete hotels

Create Hotel

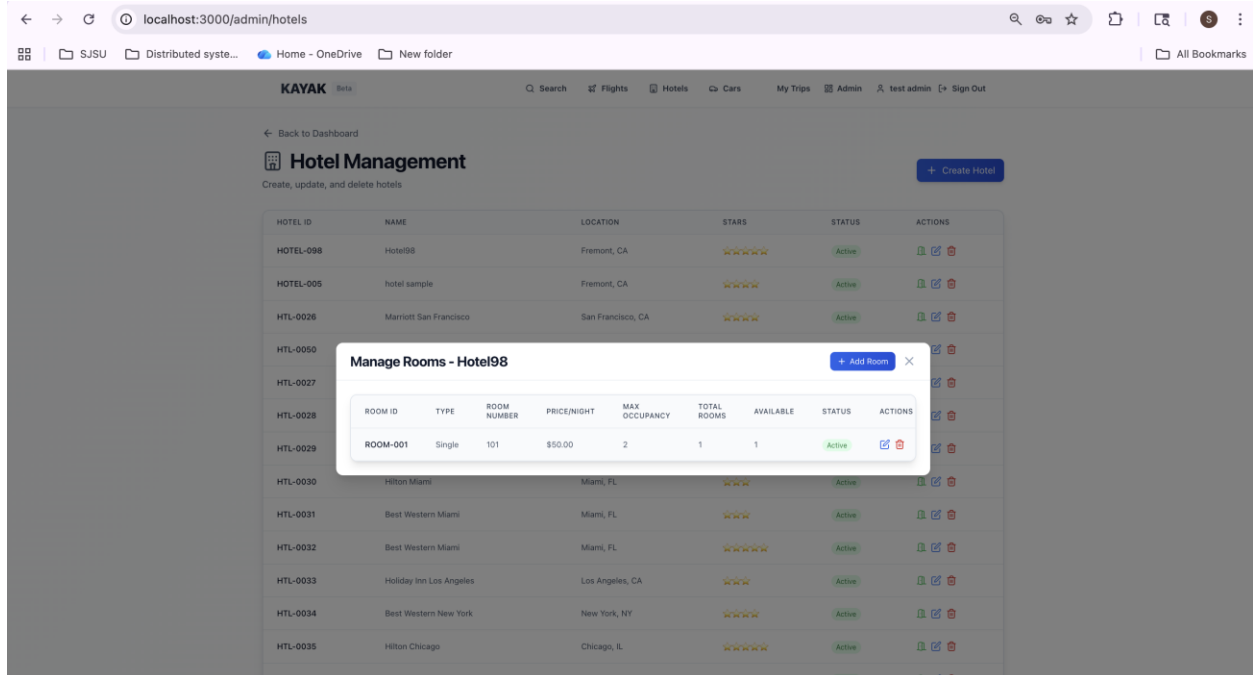
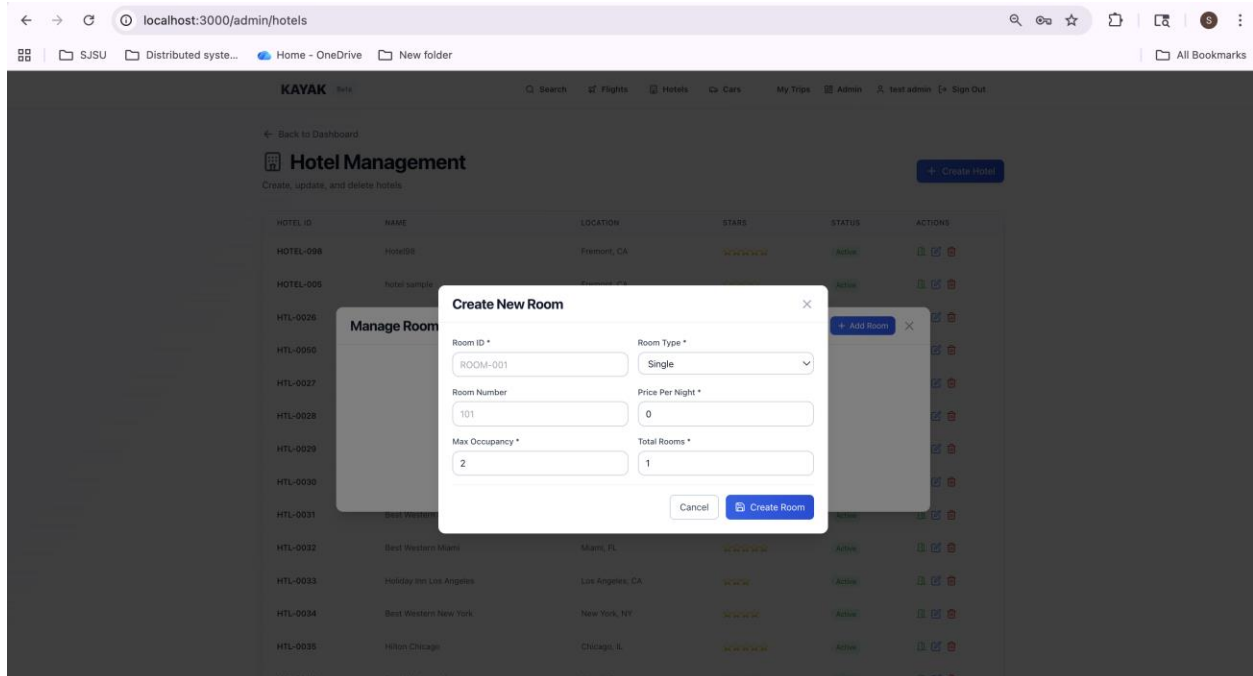
HOTEL ID	NAME	LOCATION	STARS	STATUS	ACTIONS
HOTEL-098	Hotel98	Fremont, CA	★★★★★	Active	
HOTEL-005	hotel sample	Fremont, CA	★★★★★	Active	
HTL-0026					
HTL-0050					
HTL-0027					
HTL-0028					
HTL-0029					
HTL-0030					
HTL-0031	Best Western Miami	Miami, FL	★★★★★	Active	
HTL-0032	Best Western Miami	Miami, FL	★★★★★	Active	
HTL-0033	Holiday Inn Los Angeles	Los Angeles, CA	★★★★★	Active	
HTL-0034	Best Western New York	New York, NY	★★★★★	Active	
HTL-0035	Hilton Chicago	Chicago, IL	★★★★★	Active	
HTL-0036	Best Western Miami	Miami, FL	★★★★★	Active	

Manage Rooms - Hotel98

+ Add Room

No rooms found for this hotel

Create First Room



CAR ID	VEHICLE	TYPE	LOCATION	PRICE/DAY	SPECS	STATUS	ACTIONS
CAR-0047	Honda Civic (2022)	Compact	San Francisco, CA	\$132.00	5 seats, 4 doors, manual	Unavailable	
CAR-0056	Toyota Corolla (2020)	Van	Los Angeles, CA	\$100.00	4 seats, 4 doors, automatic	Available	
CAR-0055	Toyota Corolla (2022)	Compact	New York, NY	\$111.00	4 seats, 4 doors, manual	Available	
CAR-0054	Toyota Corolla (2022)	Sedan	San Francisco, NY	\$104.00	7 seats, 4 doors, manual	Available	
CAR-0053	Honda Accord (2022)	Compact	Los Angeles, NY	\$42.00	4 seats, 4 doors, manual	Unavailable	
CAR-0052	Honda Accord (2021)	Compact	San Francisco, CA	\$107.00	7 seats, 4 doors, automatic	Available	
CAR-0051	Ford Fusion (2024)	Compact	New York, CA	\$107.00	4 seats, 4 doors, manual	Available	
CAR-0050	Toyota Corolla (2024)	Suv	Los Angeles, NY	\$132.00	4 seats, 4 doors, manual	Available	
CAR-0049	Toyota Camry (2025)	Luxury	Los Angeles, CA	\$98.00	5 seats, 4 doors, manual	Available	
CAR-0048	Toyota Corolla (2024)	Suv	New York, NY	\$121.00	4 seats, 4 doors, manual	Available	
CAR-0057	Ford Escape (2025)	Compact	San Francisco, CA	\$88.00	5 seats, 4 doors, automatic	Available	
CAR-0046	Ford Escape (2025)	Van	Los Angeles, NY	\$144.00	4 seats, 4 doors, manual	Available	
CAR-0045	Honda Accord (2023)	Sedan	New York, CA	\$141.00	7 seats, 4 doors, automatic	Available	
CAR-0044	Honda Accord (2023)	Suv	Los Angeles, CA	\$51.00	7 seats, 4 doors, automatic	Available	

Module 10: AI Concierge Testing

Test 10.1: Get Deals

Objective: Verify deals API

Steps:

1. Access deals endpoint

Expected Result:

- Deals returned
- Deal scores calculated

API Test:

curl "http://localhost:8008/api/deals?listing_type=flight&min_score=50&limit=20"

Test 10.2: Find Trip Bundles

Objective: Verify bundle recommendations

Steps:

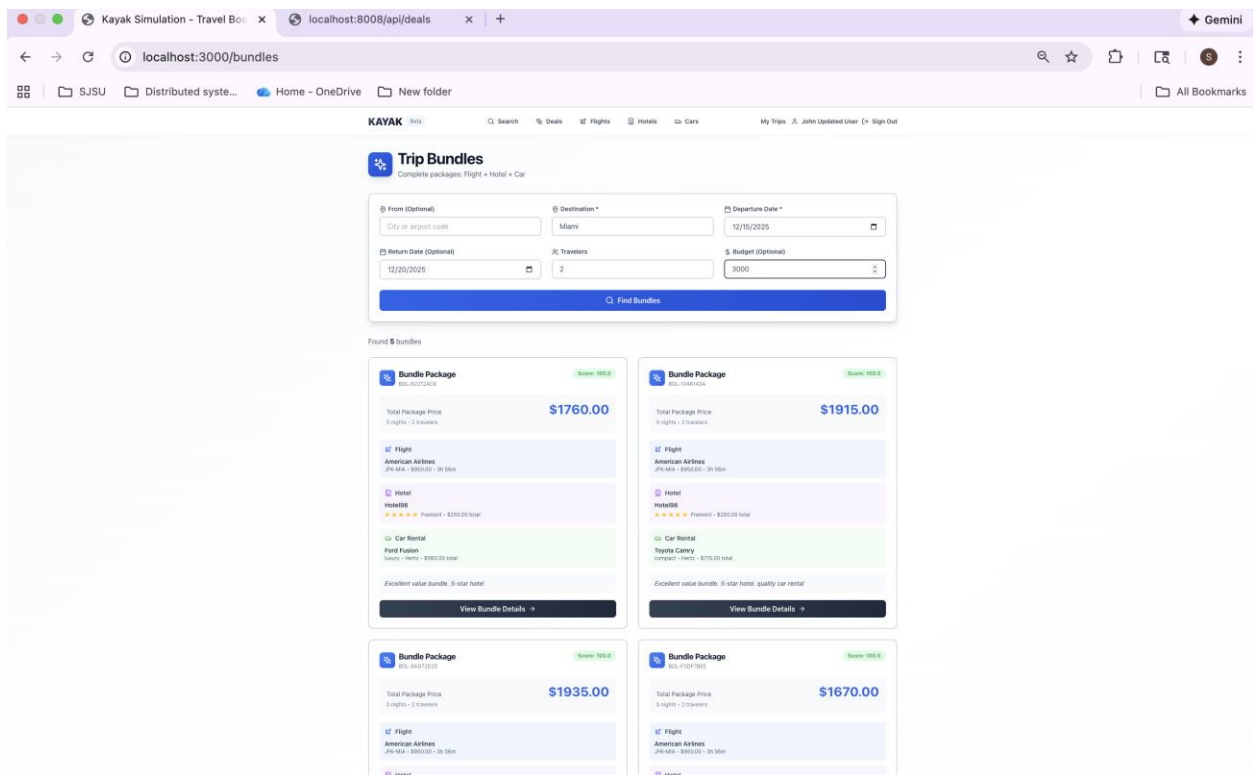
1. Request trip bundle

Expected Result:

-  Bundles recommended
-  Flight + Hotel + Car combinations

API Test:

```
curl -X POST http://localhost:8008/api/bundles \  
-H "Content-Type: application/json" \  
-d '{  
  "destination": "Miami",  
  "check_in": "2025-12-15",  
  "check_out": "2025-12-20",  
  "num_guests": 2  
}
```



Test 10.3: Chat with Concierge

Objective: Verify chatbot functionality

Steps:

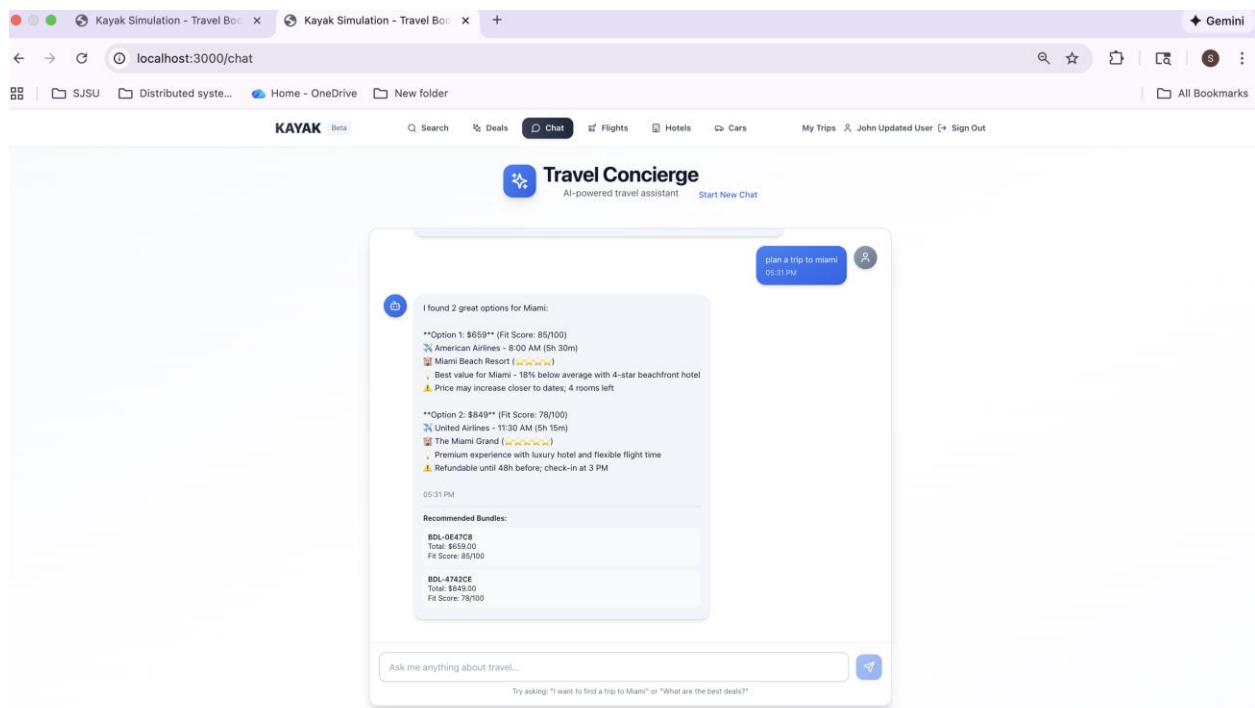
1. Open chat interface
2. Ask travel question
3. Receive response

Expected Result:

- ☒ Chat responds
- ☒ Conversation history stored in MongoDB

API Test:

```
curl -X POST http://localhost:8008/api/chat \
-H "Content-Type: application/json" \
-d '{
  "user_id": "123-45-6789",
  "message": "What are the best hotels in San Francisco?"
}'
```



Test 10.4: Real-time Deal Updates (WebSocket)

Objective: Verify WebSocket events

Steps:

1. Connect to WebSocket: `ws://localhost:8008/ws/events`
2. Wait for deal notifications

Expected Result:

- ☒ WebSocket connection established
- ☒ Deal notifications received

Module 11: Event-Driven Kafka Testing

Test 11.1: User Created Event

Objective: Verify user.created event published

Steps:

1. Create user
2. Check Kafka topic

Expected Result:

-  Event published to `kayak.user.created`
-  Event consumed by services

Validation:

docker-compose exec kafka kafka-console-consumer --bootstrap-server localhost:9092 --topic kayak.user.created --from-beginning --max-messages 1

```
(base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % docker-compose exe
c kafka kafka-topics --bootstrap-server localhost:9092 --list
__consumer_offsets
kayak.ai.deal.events
kayak.ai.deals.normalized
kayak.ai.deals.scored
kayak.ai.deals.tagged
kayak.ai.raw_supplier_feeds
kayak.analytics.click
kayak.analytics.pageview
kayak.analytics.search
kayak.booking.cancelled
kayak.booking.completed
kayak.booking.created
kayak.booking.updated
kayak.listing.car.created
kayak.listing.car.updated
kayak.listing.flight.created
kayak.listing.flight.updated
kayak.listing.hotel.created
kayak.listing.hotel.updated
kayak.notification.email
kayak.notification.push
kayak.notification.sms
kayak.payment.completed
kayak.payment.failed
kayak.payment.initiated
kayak.payment.refunded
kayak.review.created
kayak.review.deleted
kayak.review.updated
kayak.search.car
kayak.search.flight
kayak.search.hotel
kayak.user.created
kayak.user.deleted
kayak.user.updated
(base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation %
```

```
(base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % docker-compose exe
c mongodb mongosh -u admin -p kayak_mongo_pass --authenticationDatabase admin --eval
"db.adminCommand('ping')" kayak_db
{ ok: 1 }
```

Test 11.2: Booking Created Event

Objective: Verify booking.created event

Steps:

1. Create booking
2. Check Kafka topic

Expected Result:

-  Event published
-  Email service consumes event

```
(base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % curl -X POST "http://localhost:8001/users" \
-H "Content-Type: application/json" \
-d '{
  "user_id": "456-35-4001",
  "first_name": "Test",
  "last_name": "User",
  "email": "testuser001@example.com",
  "password": "testpass123",
  "phone_number": "555-0100",
  "city": "San Francisco",
  "state": "CA",
  "zip_code": "94102"
}'
{"user_id": "456-35-4001", "first_name": "Test", "last_name": "User", "email": "testuser001@example.com", "phone_number": "555-0100", "address": null, "city": "San Francisco", "state": "CA", "zip_code": "94102", "profile_image_url": null, "credit_card_last_four": null, "credit_card_type": null, "is_active": true, "created_at": "2025-12-06T02:04:23", "updated_at": "2025-12-06T02:04:23"}

(base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % docker-compose exec kafka kafka-console-consumer \
--bootstrap-server localhost:9092 \
--topic kayak.user.created \
--from-beginning \
--max-messages 1
{"event_type": "created", "data": {"flight_id": "AA765", "route": "SFO-JFK"}, "metadata": {"timestamp": "2025-12-05T13:09:13.526600", "source": "kayak-backend"}}
Processed a total of 1 messages

(base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation %

(base) savithavijayarangan@Savithas-MacBook-Air kayak-simulation % docker-compose logs user-service --tail 20 | grep -i "user\|event\|publish"
kayak_user_service | INFO:sqlalchemy.engine.Engine:ROLLBACK
kayak_user_service | INFO: 127.0.0.1:59372 - "GET /health HTTP/1.1" 200 OK
kayak_user_service | 2025-12-06 02:06:23,823 INFO sqlalchemy.engine.Engine BEGIN (implicit)
kayak_user_service | INFO:sqlalchemy.engine.Engine:BEGIN (implicit)
kayak_user_service | 2025-12-06 02:06:23,825 INFO sqlalchemy.engine.Engine SELECT 1
kayak_user_service | INFO:sqlalchemy.engine.Engine:SELECT 1
kayak_user_service | 2025-12-06 02:06:23,826 INFO sqlalchemy.engine.Engine [cached since 2.73e+04s ago] {}
kayak_user_service | INFO:sqlalchemy.engine.Engine:[cached since 2.73e+04s ago] {}
kayak_user_service | INFO:sqlalchemy.engine.Engine:ROLLBACK
kayak_user_service | 2025-12-06 02:06:23,828 INFO sqlalchemy.engine.Engine ROLLBACK
kayak_user_service | INFO: 127.0.0.1:42342 - "GET /health HTTP/1.1" 200 OK
kayak_user_service | 2025-12-06 02:06:33,956 INFO sqlalchemy.engine.Engine BEGIN (implicit)
kayak_user_service | INFO:sqlalchemy.engine.Engine:BEGIN (implicit)
kayak_user_service | INFO:sqlalchemy.engine.Engine:SELECT 1
kayak_user_service | 2025-12-06 02:06:33,957 INFO sqlalchemy.engine.Engine SELECT 1
kayak_user_service | 2025-12-06 02:06:33,958 INFO sqlalchemy.engine.Engine [cached since 2.731e+04s ago] {}
kayak_user_service | INFO:sqlalchemy.engine.Engine:[cached since 2.731e+04s ago] {}
kayak_user_service | 2025-12-06 02:06:33,959 INFO sqlalchemy.engine.Engine ROLLBACK
kayak_user_service | INFO:sqlalchemy.engine.Engine:ROLLBACK
kayak_user_service | INFO: 127.0.0.1:46720 - "GET /health HTTP/1.1" 200 OK
```

Test 11.3: Payment Completed Event

Objective: Verify payment.completed event

Steps:

1. Process payment
2. Check Kafka topic

Expected Result:

-  Event published
-  Email service consumes event

Test 11.4: Kafka Consumer Groups Active

Objective: Verify consumers running

Steps:

1. List consumer groups

Expected Result:

-  email-service-group active
-  Other consumer groups active

Validation:

docker-compose  kafka kafka-consumer-groups --bootstrap-server localhost:9092 --list

Test 11.5: Event Processing

Objective: Verify events processed correctly

Steps:

1. Trigger events
2. Verify downstream effects

Expected Result:

-  Events processed
-  Services react correctly

Module 12: MongoDB Analytics Testing

Test 12.1: Clickstream Logging

Objective: Verify user actions logged

Steps:

1. Perform actions (search, click, filter)
2. Check MongoDB logs collection

Expected Result:

-  Actions logged
-  Data stored in MongoDB

Validation:

docker-compose exec mongodb mongosh -u admin -p kayak_mongo_pass --authenticationDatabase admin --eval "db.logs.find().limit(5).pretty()"

Test 12.2: User Preferences

Objective: Verify preferences tracked

Steps:

1. User performs searches/bookings
2. Check preferences collection

Expected Result:

-  Preferences built automatically
-  Stored in MongoDB

Test 12.3: Price History

Objective: Verify price tracking

Steps:

1. Check price history collection

Expected Result:

-  Price changes tracked
-  Historical data stored

Test 12.4: Reviews Storage

Objective: Verify reviews stored

Steps:

1. Create review
2. Check MongoDB

Expected Result:

-  Review stored
-  Images stored (if applicable)

Test 12.5: Admin Audit Logs

Objective: Verify audit logs stored

Steps:

1. Admin performs action
2. Check audit logs

Expected Result:

-  Logs stored in MongoDB
-  Complete audit trail

Module 13: System Infrastructure Testing

Test 13.1: Docker Container Health

Objective: Verify all containers healthy

Steps:

1. Check container status

Expected Result:

-  All containers running
-  Health checks passing

Validation:

`docker-compose ps`

Test 13.2: Service Communication

Objective: Verify inter-service communication

Steps:

1. Test service-to-service calls
2. Verify network connectivity

Expected Result:

-  Services communicate
-  Network working

Test 13.3: Database Connections

Objective: Verify database connectivity

Steps:

1. Test MySQL connection
2. Test MongoDB connection
3. Test Redis connection

Expected Result:

-  All databases accessible
-  Connections stable

Test 13.4: CORS Configuration

Objective: Verify CORS enabled

Steps:

1. Make API call from frontend
2. Check CORS headers

Expected Result:

-  CORS headers present
-  Frontend can call APIs

Team Contribution

This project was successfully completed through the collaborative efforts of all team members. Each member contributed equally across different domains, ensuring balanced participation and skill application. The following table outlines the primary responsibilities assigned to each team member.

1. Anshika Goel (018317819)

- Led the development of the backend microservices architecture using Node.js and Express.
- Implemented User Service and Admin Service including authentication, authorization, and CRUD APIs.
- Integrated Kafka event streaming for backend communication.
- Contributed to preparing the system architecture documentation and coordinated technical decisions.
- Supported Kubernetes deployment and containerization tasks.

2. Savitha Vijayarangana (018315986)

- Developed the React-based frontend, including user-facing pages for hotel, flight, and car booking.
- Worked on Admin dashboard UI, charts, and analytics visualization.
- Implemented API integration between frontend and microservices.
- Conducted UI testing and ensured design consistency across modules.
- Assisted in preparing frontend workflow documentation for the final report.

3. Sujith Dugyala (018322694)

- Developed backend services including Car Service and Hotel Service using Express.js.
- Managed MySQL database design, schema creation, and relational mapping for booking and inventory modules.
- Implemented Redis caching mechanisms for faster data retrieval.
- Performed unit testing of services and optimized query performance.
- Contributed to deployment configuration in Kubernetes.

4. Daniel Kim (014641497)

- Developed the Flights Service and contributed to backend API orchestration logic.
- Configured and managed MongoDB database, including collections for user profiles and AI data.
- Implemented data validation, serialization, and indexing strategies.
- Worked with Kafka consumers/producers for event processing.
- Assisted with logging & monitoring setup using Kubernetes tools.

5. Lakshmi Bharathy Kumar (017613414)

- Developed the AI Service using Python FastAPI, responsible for personalized itinerary generation.
- Created algorithms to analyze booking data and user preferences from MongoDB.
- Tested and optimized AI response times and ensured compatibility with other microservices.
- Prepared documentation explaining the AI workflow and model logic.
- Supported integration of the AI API with the main backend and frontend.

6. Liza Bharatkumar Lad (018292703)

- Led performance testing using Apache JMeter, including load, stress, and endurance tests.
- Collected and analyzed metrics such as throughput, latency, and error rates.
- Documented performance test results, bottlenecks, and recommendations.
- Contributed to preparing the final project report, including diagrams and formatting.
- Assisted with AWS setup and validation of deployed services.

Collaborative Contributions

All team members jointly contributed to:

- Idea brainstorming and system design
- Creating UML diagrams, sequence diagrams, and architecture flow
- Preparing the project presentation and report
- Conducting final testing and debugging
- Performing deployment verification on AWS

Observations and Lessons Learned

Throughout the development of this cloud-based travel booking system, several key observations were made that significantly contributed to our understanding of modern software engineering practices. One of the most notable observations was the importance of **modular architecture** in large-scale applications. By dividing the system into independent microservices, we were able to achieve clearer separation of responsibilities, easier debugging, and flexible scalability. This architecture also highlighted the complexities involved in service orchestration and communication, particularly when integrating tools like **Kafka**, **Redis**, and **FastAPI** alongside Node.js services.

We also observed the critical role of **cloud-native deployment** using Docker and Kubernetes. Containerization provided environmental consistency across development, testing, and production, while Kubernetes offered powerful capabilities for load balancing, automatic scaling, and failure recovery. These tools reinforced the importance of automation and infrastructure-as-code in modern deployment pipelines. The integration with AWS further emphasized the need for proper configuration, security practices, and resource monitoring to maintain system stability.

Performance testing with **JMeter** revealed practical insights into how a distributed system behaves under real-world load. We learned that caching layers such as Redis drastically improve system responsiveness, while message brokers like Kafka enable efficient asynchronous communication. We also learned that backend services must be optimized not only for functionality but also for latency, throughput, and fault tolerance, especially when handling complex workflows like booking and AI-driven itinerary generation.

Overall, the project taught us valuable lessons in teamwork, version control, and collaborative development. Communication and coordination were essential for aligning microservice interfaces, ensuring consistent API design, and synchronizing deployment efforts. The experience reinforced the importance of planning, documenting architecture, and conducting iterative testing throughout the development cycle. Ultimately, this project strengthened our skills in full-stack development, cloud computing, distributed systems, and performance engineering—providing practical knowledge that will be highly applicable in future real-world software projects.